Agradecimientos

A mi tutor, Albert Sanchís, por guiarme durante todo el camino: por sus correos con la tarea siguiente cuando el resultado anterior aún estaba fresco.

Resumen

Los modelos fundacionales de reconocimiento del habla transcriben con gran calidad grabaciones completas en decenas de idiomas, pero muchas aplicaciones, como el subtitulado en directo o el seguimiento de debates en otra lengua, necesitan el texto mientras se habla. Este trabajo estudia qué ocurre con la calidad y con el coste cuando se exige a estos modelos transcribir en tiempo real, sobre las intervenciones en alemán, castellano, francés e inglés del corpus Europarl-ST, extraídas de los debates del Parlamento Europeo.

Se evalúan dos modelos de pesos abiertos con estrategias opuestas: Voxtral Mini 4B Realtime, un modelo causal diseñado para el streaming que emite texto con un retardo configurable, y Qwen3-ASR 1.7B, cuyo modo en streaming procesa el audio por bloques y vuelve a analizar en cada paso todo lo recibido. Los experimentos, ejecutados en una tarjeta gráfica doméstica de 12 GB con PyTorch, transformers y vLLM, miden la tasa de error de palabra en diferido sobre el test completo, la latencia algorítmica y computacional y su evolución a lo largo de cada intervención en un barrido de latencias, y la degradación al pasar del modo diferido al streaming en los cuatro idiomas.

En diferido, ambos modelos mejoran la línea base publicada con el corpus y, descontados unos pocos fallos catastróficos, ofrecen una precisión equivalente. En tiempo real el panorama cambia. A igual latencia algorítmica de 480 ms, Voxtral mantiene constante el coste de cada paso, no pierde calidad respecto a su modo diferido y no presenta incidencias en castellano, mientras que el coste por paso de Qwen3-ASR crece con la duración de la intervención, supera el tiempo real en las intervenciones largas y agota su contexto en algunas de ellas; con bloques de varios segundos, en cambio, Qwen3-ASR es muy preciso y estable. El factor de tiempo real, la medida habitual de la velocidad, ordena los dos modelos al revés que la latencia, por lo que se propone evaluar los sistemas en directo combinando el uso del presupuesto de tiempo de cada paso con su crecimiento.

El trabajo se completa con un demostrador web, construido con Gradio, que transcribe en tiempo real la voz del usuario y las intervenciones del corpus con cualquiera de los dos modelos, muestra el coste en directo y puede compartirse con acceso restringido sin coste económico.

Palabras clave: transcripción automática del habla, modelos fundacionales, tiempo real, latencia, multilingüismo, Europarl-ST.

Abstract

Speech foundation models transcribe complete recordings in dozens of languages with high accuracy, but many applications, such as live captioning or following a debate held in another language, need the text while people are still speaking. This work studies what happens to accuracy and cost when these models are required to transcribe in real time, using the German, English, French and Spanish speeches of the Europarl-ST corpus, taken from the debates of the European Parliament.

Two open-weight models with opposite strategies are evaluated: Voxtral Mini 4B Realtime, a causal model designed for streaming that emits text after a configurable delay, and Qwen3-ASR 1.7B, whose streaming mode processes the audio in chunks and re-analyses everything received so far at every step. The experiments, run on a 12 GB consumer graphics card with PyTorch, transformers and vLLM, measure the word error rate in offline mode on the full test set, the algorithmic and computational latency and its evolution along each speech in a latency sweep, and the degradation from offline to streaming mode in the four languages.

In offline mode, both models improve on the baseline published with the corpus and, once a few catastrophic failures are discounted, deliver equivalent accuracy. In real time the picture changes. At the same algorithmic latency of 480 ms, Voxtral keeps the cost of every step constant, loses no accuracy with respect to its offline mode and shows no incidents in Spanish, whereas the per-step cost of Qwen3-ASR grows with the length of the speech, falls behind real time on long speeches and exhausts its context on some of them; with chunks of several seconds, however, Qwen3-ASR is very accurate and stable. The real-time factor, the usual measure of speed, ranks the two models in the opposite order to latency, so this work proposes evaluating live systems by combining the share of each step's time budget that is used with how that share grows.

The work is completed with a web demonstrator, built with Gradio, that transcribes the user's voice and the corpus speeches in real time with either model, displays the cost live and can be shared with restricted access at no monetary cost.

Keywords: automatic speech recognition, foundation models, real time, latency, multilingualism, Europarl-ST.

Índice

Agradecimientos 1
Resumen 2
Abstract 3
1. Introducción 9
1.1. Motivación 9
1.2. Objetivos 9
1.3. Impacto esperado 10
1.4. Metodología y plan de trabajo 11
1.5. Estructura de la memoria 12
1.6. Convenciones 12
2. Fundamentos 13
2.1. El reconocimiento automático del habla 13
2.1.1. De la onda al espectrograma log-Mel 13
2.1.2. Del modelo híbrido al modelo de extremo a extremo 14
2.2. El Transformer 15
2.2.1. Atención: consultas, claves y valores 15
2.2.2. Codificador, decodificador y generación autorregresiva 16
2.2.3. Tokens y vocabulario 17
2.2.4. La arquitectura de los decodificadores actuales 17
2.3. Modelos fundacionales del habla 18
2.4. Reconocimiento en tiempo real 19
2.4.1. Causalidad y procesamiento incremental 19
2.4.2. Adaptar modelos diferidos o diseñar modelos nativos 19
2.4.3. Latencia algorítmica, computacional y total 20
2.4.4. Hipótesis definitivas y revisables 20
2.5. Métricas 21
2.5.1. Tasa de error de palabra y normalización 21
2.5.2. Factor de tiempo real 22
2.5.3. Uso del paso y crecimiento 22
3. Modelos fundacionales de ASR y conjunto de datos Europarl-ST 24
3.1. Criterios de selección 24
3.2. Voxtral Mini 4B Realtime 24
3.2.1. Arquitectura 24
3.2.2. El retardo como condición del modelo 25
3.2.3. Resultados publicados 26

3.3. Qwen3-ASR 1.7B 27
3.3.1. Arquitectura y entrenamiento 27
3.3.2. Streaming por reprocesado 27
3.4. Dos maneras de entender el tiempo real 29
3.5. El corpus Europarl-ST 29
3.5.1. Origen y construcción 29
3.5.2. La partición de test empleada 30
3.5.3. La línea base del artículo original 31
3.6. Crítica del estado del arte y propuesta 31
4. Evaluación experimental 32
4.1. Entorno experimental 32
4.2. Metodología común 33
4.3. E1: transcripción en diferido del test completo 33
4.3.1. Resultados por idioma 33
4.3.2. Fallos catastróficos 34
4.3.3. Distribución y tipos de error 35
4.3.4. El caso del inglés 36
4.3.5. Coste en diferido 37
4.4. E2: barrido de latencia en castellano 37
4.4.1. Diseño 37
4.4.2. Voxtral: coste constante y una banda fiable 38
4.4.3. Qwen3-ASR y el techo de contexto 39
4.4.4. Comparación a igual latencia 40
4.4.5. Variabilidad del tiempo de cómputo 41
4.5. E3: diferido frente a streaming en los cuatro idiomas 42
4.5.1. Diseño y muestreo 42
4.5.2. Degradación por idioma 42
4.5.3. Incidencias 45
4.6. Discusión 45
4.6.1. Lo que mide cada métrica 45
4.6.2. Limitaciones 46
4.6.3. Implicaciones para un sistema en directo 46
5. Desarrollo del demostrador 48
5.1. Requisitos 48
5.2. Soluciones posibles y solución elegida 48
5.3. Arquitectura 49

5.4. Implementación 50
5.4.1. La sesión de streaming de Voxtral 50
5.4.2. Qwen3-ASR en un proceso aparte 52
5.4.3. Interfaz 52
5.4.4. Concurrencia, sesiones y cambio de modelo 53
5.5. Pruebas 54
5.6. Limitaciones, riesgos y marco legal 55
6. Conclusiones y trabajo futuro 56
6.1. Cumplimiento de los objetivos 56
6.2. Problemas, errores y lecciones aprendidas 57
6.3. Relación del trabajo con los estudios cursados 58
6.4. Trabajo futuro 59
Bibliografía 61
Anexo A. Glosario 64
Anexo B. Presupuesto 66
Anexo C. Objetivos de Desarrollo Sostenible 68

Índice de figuras

Figura 1. Onda (arriba) y espectrograma log-Mel de 128 bandas (abajo) de los primeros cuatro segundos de la intervención en.20100210.29.3-529 del test de castellano de Europarl-ST, que comienza con «Señor Presidente». Ventanas de 25 ms con saltos de 10 ms. Elaboración propia. 14
Figura 2. Esquema de un Transformer de codificador y decodificador aplicado al reconocimiento del habla. Cada subcapa va acompañada de una conexión residual y una normalización, que en el decodificador se omiten por claridad. Elaboración propia a partir de Vaswani et al. (2017). 16
Figura 3. Componentes de la latencia percibida al transcribir una palabra en tiempo real. Elaboración propia. 20
Figura 4. Esquema de emisión de Voxtral Realtime con un retardo τ de 240 ms (tres tramas). Cada casilla es una trama de 80 ms. La segmentación de las palabras en tokens es ilustrativa. Elaboración propia a partir de Mistral AI (2026). 26
Figura 5. Audio que procesa cada modelo en cada paso (izquierda) y acumulado (derecha, escala logarítmica) a lo largo de una intervención de 120 s. Las curvas se calculan a partir del mecanismo de cada modelo, no de mediciones. Elaboración propia. 28
Figura 6. WER en diferido de los dos modelos por idioma y línea base del artículo de Europarl-ST. Elaboración propia; línea base tomada de Iranzo-Sánchez et al. (2020). 34
Figura 7. Distribución del WER por intervención en diferido. Las cajas abarcan del primer al tercer cuartil, la línea blanca es la mediana y los círculos son las intervenciones por encima del 30 %; los valores superiores al 60 % se dibujan en el borde y se indica su número. Elaboración propia a partir de las salidas de E1. 35
Figura 8. WER sin anómalos frente a latencia total media de cada configuración en castellano. La línea de puntos une el último valor admitido de Voxtral con el valor fuera de especificación. Elaboración propia. 40
Figura 9. Uso del paso en el último tercio de cada intervención frente a su duración, a 480 ms de latencia algorítmica. Las cruces marcan las intervenciones en las que Qwen3-ASR agotó el contexto. Elaboración propia a partir de los tiempos medidos en E2. 41
Figura 10. WER en diferido y en streaming a 480 ms de los dos modelos en los cuatro idiomas. Elaboración propia. 43
Figura 11. WER en streaming frente a WER en diferido de cada intervención de E3. Los puntos sobre la diagonal empeoran al pasar a streaming. Los valores superiores al 40 % se dibujan en el borde. Elaboración propia. 44
Figura 12. Arquitectura del demostrador. Voxtral se ejecuta dentro del proceso de la aplicación; Qwen3-ASR, en un proceso aparte con su propio entorno. Elaboración propia. 50
Figura 13. Secuencia de una sesión de transcripción con Voxtral. Los mensajes del hilo del modelo se repiten cada 80 ms de audio; los del navegador, cada 0,2 s. Elaboración propia. 51
Figura 14. Interfaz del demostrador en la pestaña de audios de Europarl-ST, con la transcripción de Voxtral a 480 ms, la referencia del corpus y las métricas de la sesión. Elaboración propia. 53

Índice de tablas

Tabla 1. Criterios de cumplimiento de los objetivos específicos. 10
Tabla 2. Plan de trabajo seguido y horas estimadas. 11
Tabla 3. Alineamiento de dos fragmentos de una transcripción de Voxtral con su referencia. Sobre las 11 palabras de referencia de los dos fragmentos hay una sustitución y dos inserciones: un WER del 27,27 %. 21
Tabla 4. Configuración de Voxtral Realtime. Fuente: Mistral AI (2026). 25
Tabla 5. WER (%) en FLEURS de Voxtral Realtime según el retardo, frente a Whisper en diferido. Fuente: Mistral AI (2026), tabla 7. 26
Tabla 6. Tasa de error (%) de Qwen3-ASR 1.7B en diferido y en streaming según su informe técnico. Fuente: Shi et al. (2026), tabla 8. 28
Tabla 7. Comparación de los dos modelos evaluados. Elaboración propia a partir de Mistral AI (2026), Shi et al. (2026) y de sus fichas oficiales. 29
Tabla 8. Partición de test usada en cada idioma y WER de los sistemas de referencia del artículo original. Fuente de la última columna: Iranzo-Sánchez et al. (2020), tabla 4. 30
Tabla 9. Rasgos de la partición de test de cada idioma. Las medias y la velocidad se calculan a partir de las horas y palabras de la tabla 8. 30
Tabla 10. Versiones del software empleado en cada entorno. 32
Tabla 11. WER en diferido sobre el test completo de Europarl-ST. 33
Tabla 12. WER en diferido sin los audios con transcripción anómala (diagnóstico). . 35
Tabla 13. Sustituciones (S), eliminaciones (D) e inserciones (I) en E1, en porcentaje sobre las palabras de referencia. La suma de cada terna es el WER de la tabla 11. 36
Tabla 14. Descomposición del WER de Voxtral en inglés en E1. Los efectos se estimaron por separado y su suma no coincide exactamente con la diferencia total, porque interactúan. 36
Tabla 15. Tiempo de proceso en diferido en Windows y RTF por idioma. 37
Tabla 16. Barrido de latencia de Voxtral sobre veinte intervenciones en castellano (WSL2). 38
Tabla 17. Barrido de latencia de Qwen3-ASR sobre las mismas veinte intervenciones (WSL2). Los fallos quedan fuera del WER y de las latencias. 39
Tabla 18. Comparación de los dos modelos a 480 ms de latencia algorítmica. 40
Tabla 19. Tiempo de cómputo por paso en E2: media, percentil 90 y medias del primer y del último tercio de cada intervención, promediados sobre las intervenciones completadas. 41
Tabla 20. Muestra de E3: veinte intervenciones por idioma de hasta 90 s. 42
Tabla 21. WER en diferido y en streaming a 480 ms, y degradación en puntos. El streaming de Qwen3-ASR en inglés se calcula sobre 19 intervenciones. 42
Tabla 22. RTF medio por intervención en E3 (WSL2). 45
Tabla 23. Requisitos del demostrador. 48
Tabla 24. Alternativas para el despliegue del demostrador. Precios y condiciones consultados el 19-10-2026. 48
Tabla 25. Pruebas del demostrador sobre la RTX 3060. 54
Tabla 26. Riesgos del demostrador y medidas adoptadas. 55
Tabla 27. Presupuesto del trabajo 67
Tabla 28. Grado de relación del trabajo con los ODS. 68

1. Introducción

1.1. Motivación

Hay palabras que solo sirven en el momento en que se pronuncian. Un subtítulo que llega diez segundos tarde a una retransmisión ya no acompaña a la imagen; la transcripción de un debate que se publica al día siguiente no permite seguirlo mientras sucede. Para quien no oye, o para quien no entiende la lengua en la que se habla, la diferencia entre transcribir después y transcribir mientras tanto es la diferencia entre leer el acta y estar presente.

El reconocimiento automático del habla ha cambiado de escala en pocos años. Los grandes proveedores tecnológicos publican modelos preentrenados con cientos de miles de horas de audio, los llamados modelos fundacionales, que transcriben decenas de idiomas sin necesidad de entrenarlos para cada uno (Bommasani et al. 2021; Radford et al. 2023). En la transcripción de grabaciones completas su calidad es muy alta. Pero la mayoría de esos modelos se concibieron para escuchar un fichero entero antes de escribir la primera palabra, y la pregunta de qué ocurre cuando se les exige transcribir mientras se habla no tiene una respuesta única. ¿Pierden calidad? ¿Cuánta? ¿Les alcanza el hardware disponible para seguir el ritmo de la voz durante una intervención de varios minutos? Este trabajo responde a esas preguntas para dos modelos concretos, y la respuesta corta, que el capítulo 4 detalla, es que depende menos de la calidad de cada modelo que de la manera en que trata el tiempo.

Los debates del Parlamento Europeo son un banco de pruebas especialmente exigente para esas preguntas. Se celebran en varias lenguas, con intervenciones largas, vocabulario institucional y nombres propios de todo el continente, y su contenido interesa a ciudadanos que hablan otras lenguas. El grupo MLLP de la Universitat Politècnica de València construyó con ellos un corpus público, Europarl-ST, que reúne intervenciones en alemán, castellano, francés e inglés con sus transcripciones (Iranzo-Sánchez et al. 2020). Es el escenario elegido para este trabajo.

En lo personal, quería aplicar modelos de aprendizaje automático a tareas que fueran útiles de verdad, y no a ejemplos de manual que se resuelven una vez y se olvidan. La transcripción en tiempo real me pareció un caso claro: que un modelo funcione mientras alguien habla es lo que lo hace útil y cómodo en cualquier tarea que exija una respuesta inmediata, y la traducción de las intervenciones de un parlamento, donde esperar al final del discurso no sirve de nada, es el ejemplo que me convenció. Me atraía también dar el paso desde evaluar un modelo en un cuaderno hasta dejar algo que otra persona pudiera usar. Y, a medida que avanzaba el trabajo, descubrí que lo que más me interesaba no era la cifra final de calidad, sino lo que hay detrás de ella: medir la latencia me obligó a entender cómo funciona cada modelo por dentro.

1.2. Objetivos

El objetivo general del trabajo es aplicar y evaluar modelos fundacionales de reconocimiento del habla en la transcripción en tiempo real de sesiones parlamentarias multilingües, y desarrollar un demostrador que permita usar esa transcripción en directo. Se concreta en cuatro objetivos específicos, ordenados de mayor a menor relevancia para la pregunta del trabajo:

1. **O1.** Medir el comportamiento en tiempo real de Voxtral Mini 4B Realtime y Qwen3-ASR 1.7B sobre Europarl-ST: la latencia algorítmica y computacional, el coste por paso y su evolución a lo largo de una intervención, y la degradación de la tasa de error frente al modo diferido.

2. **O2.** Evaluar la tasa de error de palabra de ambos modelos en diferido sobre el test completo de Europarl-ST en los cuatro idiomas y compararla con la línea base publicada con el corpus.

3. **O3.** Desarrollar un demostrador que transcriba en tiempo real, desde el navegador, la voz del usuario en los cuatro idiomas y las intervenciones del corpus, con elección de modelo y de latencia.

4. **O4.** Estudiar los fundamentos del reconocimiento del habla con modelos fundacionales y en tiempo real necesarios para diseñar y entender los experimentos.

Cada objetivo se acompañó de un criterio que permitiera decidir, al final del trabajo, si se había alcanzado. La tabla 1 los recoge; el capítulo 6 responde a cada uno.

Tabla 1. *Criterios de cumplimiento de los objetivos específicos.*

Objetivo	Criterio de cumplimiento
O1	Latencia total, coste por paso, crecimiento y degradación medidos para los dos modelos en al menos un punto de igual latencia algorítmica, en los cuatro idiomas
O2	WER de ambos modelos sobre las 729 intervenciones del test, comparado con la línea base del corpus y con un análisis de los errores
O3	Aplicación accesible desde un navegador remoto que transcriba en tiempo real con al menos uno de los modelos en la GPU disponible
O4	Fundamentos suficientes para explicar, y no solo describir, los resultados de O1 y O2

La pregunta que recorre los cuatro objetivos puede formularse de manera más afilada. No se trata de saber qué modelo transcribe mejor, porque en diferido la respuesta depende del idioma y de unos pocos fallos raros, sino de saber qué cambia cuando el modelo deja de tener toda la grabación delante: si su error crece, si su coste sigue cabiendo en el tiempo disponible y si lo sigue haciendo al cabo de dos minutos de discurso. Son preguntas que los artículos de los modelos responden solo en parte, como se discute en la sección 3.6, y que tienen respuestas distintas para modelos que, sobre el papel, prometen lo mismo.

1.3. Impacto esperado

El primer beneficiario de una transcripción en directo fiable es quien no puede oír lo que se dice: personas sordas o con hipoacusia que siguen una retransmisión o una sesión pública a través de subtítulos. El segundo es quien oye pero no entiende la lengua del orador; la transcripción en tiempo real es el primer eslabón de cualquier sistema de traducción simultánea automática. Ambos casos se relacionan con los Objetivos de Desarrollo Sostenible de reducción de las desigualdades y de instituciones sólidas, como se razona en el anexo C.

El trabajo aspira también a un impacto más modesto y más inmediato: ofrecer a quien tenga que elegir un modelo para transcribir en directo, con un presupuesto de hardware doméstico, una comparación medida a igual latencia y con el coste a la vista. Sus resultados muestran que la métrica que suele publicarse para describir la velocidad

de un reconocedor puede ordenar los modelos justo al revés de como se comportan en directo, y esa advertencia es útil más allá de los dos modelos evaluados.

1.4. Metodología y plan de trabajo

El trabajo se organizó en etapas que se apoyan unas en otras, con revisiones periódicas con el tutor por correo electrónico y en reuniones, en las que se decidía el siguiente experimento a la vista de los resultados del anterior. Las etapas fueron cinco: estudio de los modelos y del corpus; evaluación en diferido del test completo con los dos modelos; evaluación en tiempo real, primero con un barrido de latencia en castellano y después con una comparación entre el modo diferido y el modo en *streaming* (transmisión continua: el modelo recibe el audio por fragmentos a medida que se produce) en los cuatro idiomas; desarrollo del demostrador; y redacción de esta memoria. Cada experimento se implementó en cuadernos de Jupyter que guardan las salidas en bruto de los modelos, de modo que el análisis pudiera repetirse sin volver a ejecutarlos, y sus conclusiones se recogieron en documentos de observaciones revisados con el tutor.

La tabla 2 resume el calendario real. Las horas son una estimación del trabajo efectivo y no incluyen las ejecuciones largas desatendidas, como las casi 29 horas que necesitó Voxtral para transcribir el test completo.

Tabla 2. *Plan de trabajo seguido y horas estimadas.*

Fase	Periodo (2026)	Hitos	Horas
1. Estudio y puesta en marcha	17-05 a 03-06	Repositorio, corpus y artículo de Voxtral; primer cuaderno	35
2. Evaluación en diferido con Voxtral y análisis de errores	03-06 a 13-08	Cuatro idiomas; análisis del inglés	40
3. Evaluación en diferido con Qwen3-ASR	13-08 a 08-09	Entorno propio; comparación entre modelos	30
4. Barrido de latencia y entorno WSL2	11-09 a 19-09	Cambio de métrica hacia la latencia; techo de contexto	35
5. Diferido frente a streaming en cuatro idiomas	22-09 a 24-09	Muestra aleatoria; degradación por idioma	20
6. Demostrador	13-10 a 16-10	Aplicación con Voxtral; integración de Qwen3-ASR	35
7. Memoria	19-10 a 23-11	Borrador al tutor el 9-11; depósito	75
8. Reuniones, correo y preparación de la defensa	Todo el periodo	Defensa del 9 al 11-12	30
Total			300

El plan inicial no preveía tres de las tareas que más tiempo consumieron. El entorno Linux sobre WSL2 se hizo necesario cuando se comprobó que el streaming de Qwen3-ASR exige vLLM, que no funciona en Windows. El cambio de métrica, del factor de tiempo real a la latencia, surgió en una revisión con el tutor tras la evaluación en diferido, y obligó a diseñar los experimentos de streaming. Y la integración de Qwen3-ASR en el demostrador, que no estaba prevista, exigió ejecutarlo en un proceso aparte por el conflicto de versiones entre los dos modelos. Las 300 horas quedan dentro de la dedicación que corresponde a los 12 créditos del trabajo. El presupuesto económico se detalla en el anexo B.

1.5. Estructura de la memoria

El capítulo 2 presenta los fundamentos del reconocimiento del habla necesarios para el resto: la representación del audio, la evolución de las arquitecturas hasta el Transformer, los modelos fundacionales, las particularidades del tiempo real y las métricas. El capítulo 3 describe los dos modelos evaluados y el corpus Europarl-ST, y cierra con una crítica del estado del arte y la propuesta del trabajo. El capítulo 4 recoge los tres experimentos y su discusión. El capítulo 5 describe el desarrollo del demostrador. El capítulo 6 resume el cumplimiento de los objetivos, lo aprendido, la relación del trabajo con los estudios cursados y las líneas de trabajo futuro. Tras la bibliografía, el anexo A contiene un glosario de los términos técnicos y siglas, el anexo B el presupuesto y el anexo C su vinculación con los Objetivos de Desarrollo Sostenible.

1.6. Convenciones

Los términos en otras lenguas se escriben en cursiva la primera vez que aparecen, con su traducción, y después en redonda cuando su uso en castellano es habitual, como *streaming*. Los nombres de funciones, parámetros, ficheros y fragmentos de código se escriben en fuente monoespaciada. Los decimales se separan con coma y los millares con punto a partir de cinco cifras. Las referencias bibliográficas siguen el sistema autor-año de la norma ISO 690, recomendado por la ETSINF; las páginas web de herramientas y productos se citan en nota al pie con su fecha de consulta. Las tablas y figuras sin indicación de fuente son de elaboración propia.

2. Fundamentos

Los modelos que este trabajo evalúa condensan medio siglo de ideas sobre cómo convertir una onda sonora en texto. Este capítulo recorre solo las que hacen falta para leer los capítulos siguientes: cómo se representa el audio, cómo evolucionó la arquitectura de los reconocedores hasta el Transformer, qué hace que un modelo sea fundacional, qué cambia cuando la transcripción debe producirse mientras se habla y cómo se mide todo ello. El lector objetivo es un informático que conoce el aprendizaje automático en general pero no el reconocimiento del habla; los términos técnicos se definen la primera vez que aparecen y se recogen en el glosario del anexo A.

2.1. El reconocimiento automático del habla

El reconocimiento automático del habla (ASR, por *automatic speech recognition*) busca, para una señal de audio, la secuencia de palabras que la produjo. En términos probabilísticos, dada una secuencia de vectores acústicos X, el reconocedor elige la secuencia de palabras W más probable:

$$\widehat{W} = \operatorname*{argmax}_{W} P(W \mid X) = \operatorname*{argmax}_{W} P(X \mid W)\, P(W)$$

La segunda igualdad, que aplica la regla de Bayes y descarta $P(X)$ porque no depende de W, es la que organizó el campo durante décadas: un modelo acústico estima $P(X \mid W)$, cuán verosímil es el audio si se hubieran pronunciado esas palabras, y un modelo de lenguaje estima $P(W)$, cuán verosímil es la frase en sí. Los sistemas actuales ya no separan esos dos factores, pero el problema sigue siendo el mismo: elegir, entre un número astronómico de transcripciones posibles, la que mejor explica lo que se ha oído.

2.1.1. De la onda al espectrograma log-Mel

Un micrófono entrega una sucesión de valores de presión. Para el habla basta con muestrear a 16 kHz, es decir, 16.000 valores por segundo, porque la información útil para distinguir los sonidos del habla se concentra por debajo de los 8 kHz. Los dos modelos de este trabajo esperan audio a esa frecuencia.

Una onda así es una mala entrada para un clasificador: es larga, redundante y muy sensible a factores que no cambian el mensaje, como el volumen o la fase. La representación que se usa en su lugar procede de una observación sobre el oído humano, que discrimina bien las frecuencias graves y cada vez peor las agudas. La escala Mel imita esa percepción:

$$\mathrm{mel}(f) = 1127 \ln\left(1 + \frac{f}{700}\right)$$

donde f es la frecuencia en hercios (Jurafsky y Martin 2026). El procedimiento tiene cuatro pasos. Se corta la señal en ventanas cortas y solapadas, dentro de las cuales el habla se puede considerar estable; se calcula el espectro de cada ventana con la transformada de Fourier; se agrupa la energía del espectro en un banco de filtros triangulares espaciados uniformemente en la escala Mel, más estrechos en los graves que en los agudos; y se toma el logaritmo de cada banda, que comprime el rango dinámico como lo hace el oído. La utilidad de las representaciones basadas en la escala Mel para el reconocimiento quedó establecida en un estudio comparativo clásico (Davis

y Mermelstein 1980), y sigue vigente. El resultado es el espectrograma log-Mel: una matriz con una columna por ventana y una fila por banda.

En Voxtral Realtime, según la configuración publicada con el modelo, cada ventana dura 25 ms (400 muestras), las ventanas avanzan 10 ms (160 muestras) y el banco tiene 128 filtros, de modo que cada segundo de audio se convierte en 100 columnas de 128 valores. Qwen3-ASR usa también un banco de 128 dimensiones (Shi et al. 2026). La figura 1 muestra la onda y el espectrograma de los primeros cuatro segundos de una intervención de Europarl-ST calculados con esos parámetros.

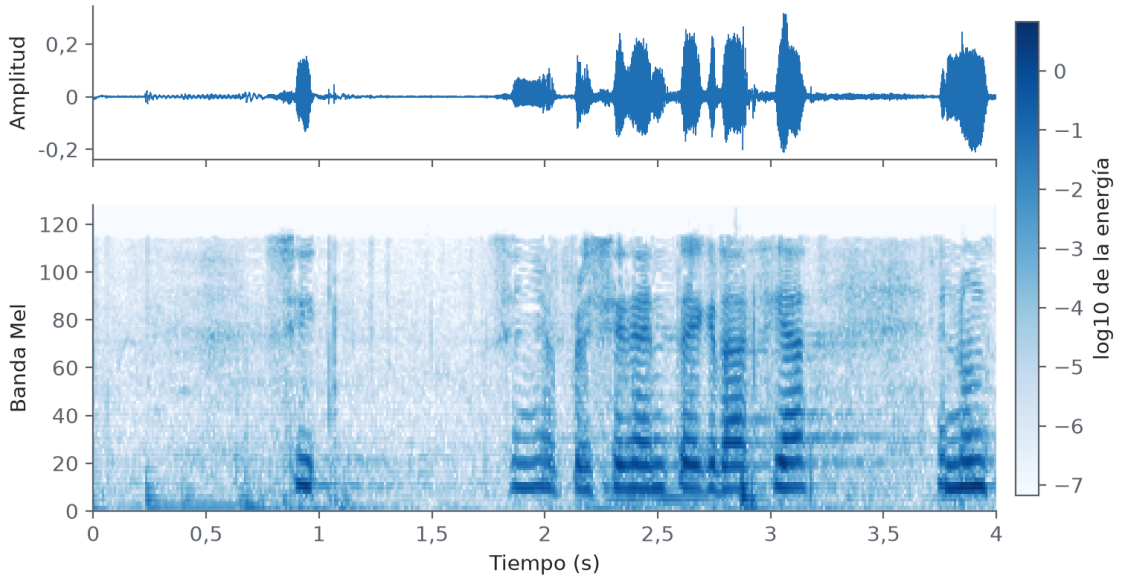


Figura 1. *Onda (arriba) y espectrograma log-Mel de 128 bandas (abajo) de los primeros cuatro segundos de la intervención en.20100210.29.3-529 del test de castellano de Europarl-ST, que comienza con «Señor Presidente». Ventanas de 25 ms con saltos de 10 ms. Elaboración propia.*

En la figura se distinguen los silencios, las vocales, que concentran la energía en bandas horizontales bien marcadas, y las consonantes fricativas, que la reparten por las bandas altas. Todo lo que el modelo sabe del audio pasa por esta matriz.

2.1.2. Del modelo híbrido al modelo de extremo a extremo

Durante más de veinte años el modelo acústico fue un modelo oculto de Markov (HMM): una máquina de estados probabilística en la que cada fonema se representa por unos pocos estados y cada estado emite vectores acústicos con cierta distribución (Rabiner 1989). Los HMM resolvían con elegancia el problema central del habla, que es la alineación: no se sabe de antemano cuántas ventanas dura cada sonido, y el algoritmo de Viterbi encuentra la segmentación más probable. Su debilidad era todo lo demás. Un reconocedor completo necesitaba un diccionario de pronunciación, un modelo de lenguaje independiente y una cadena de componentes entrenados por separado. Los sistemas híbridos sustituyeron las distribuciones de emisión por redes neuronales profundas, y con ellos se construyó la línea base de Europarl-ST que el capítulo 3 describe (Iranzo-Sánchez et al. 2020), pero conservaron la arquitectura de piezas.

La alternativa era entrenar una única red que fuera directamente del audio al texto. El obstáculo, de nuevo, era la alineación: para entrenar una red que clasifica cada ventana

hace falta saber qué carácter corresponde a cada una, y esa información no existe en los corpus, que solo contienen el audio y su transcripción. La clasificación temporal conexionista (CTC) resolvió el problema añadiendo un símbolo en blanco y definiendo la probabilidad de una transcripción como la suma de las probabilidades de todas las alineaciones compatibles con ella, calculable con programación dinámica (Graves et al. 2006). CTC permite entrenar sin alineaciones, pero supone que las salidas de cada ventana son independientes entre sí, de modo que no aprende nada sobre cómo se suceden las palabras.

Dos propuestas levantaron esa restricción. El transductor recurrente (RNN-T) añade a la red acústica una red de predicción que recibe los símbolos ya emitidos, y combina en cada paso las salidas de las dos redes para decidir el siguiente símbolo; sigue sumando sobre alineaciones, como CTC, pero condiciona cada salida a las anteriores (Graves 2012; Jurafsky y Martin 2026). Los modelos de codificador y decodificador con atención, como *Listen, Attend and Spell*, renuncian a la alineación explícita: un codificador resume el audio completo y un decodificador genera el texto carácter a carácter, eligiendo en cada paso a qué parte del audio atender (Chan et al. 2016). Sus autores subrayan que este modelo no hace ninguna suposición de independencia entre los caracteres, y esa es su ventaja sobre CTC.

La distinción importa para el tiempo real. CTC y RNN-T avanzan sobre el audio de forma monótona y se adaptan con naturalidad al streaming. El modelo de atención, en su forma original, necesita el audio completo antes de empezar a escribir. Los dos modelos de este trabajo descienden de la familia de codificador y decodificador, y cada uno resuelve a su manera esa tensión, como se vio en el capítulo 3.

2.2. El Transformer

2.2.1. Atención: consultas, claves y valores

El Transformer prescindió de la recurrencia y basó todo el procesamiento de secuencias en un mecanismo de atención (Vaswani et al. 2017). La idea es que cada posición de una secuencia construye su representación mezclando la información de las demás posiciones, con pesos que dependen del contenido y no de la distancia.

El mecanismo se formula con tres matrices. Cada posición i de la secuencia de entrada, representada por un vector x_i, se proyecta de tres maneras mediante matrices de pesos aprendidas: una consulta $q_i = x_i W^Q$, una clave $k_i = x_i W^K$ y un valor $v_i = x_i W^V$. La consulta expresa qué información busca la posición i; la clave, qué información ofrece cada posición a quien la busca; el valor, qué información entrega efectivamente si se le presta atención. La compatibilidad entre la consulta de i y la clave de j se mide con su producto escalar, y los pesos de la mezcla son esas compatibilidades normalizadas con una función softmax. Apilando los vectores en matrices Q, K y V, la operación completa es:

$$\text{Atención}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

donde d_k es la dimensión de las claves. La división entre $\sqrt{d_k}$ evita que los productos escalares crezcan con la dimensión y saturen la softmax, lo que dejaría gradientes casi

nulos. Cada fila del resultado es una media ponderada de los valores, con pesos que suman uno. Conviene precisar lo que esta fórmula no hace: no compara entradas con salidas ni guarda memoria alguna; compara cada posición con todas las demás de la secuencia que recibe y devuelve, para cada una, una combinación de lo que las otras ofrecen.

En la práctica la operación se repite en paralelo con varias proyecciones distintas, las cabezas de la atención multicabeza, y sus resultados se concatenan. Cada cabeza puede especializarse en un tipo de relación. La tabla 4 del capítulo 3 muestra que el decodificador de Voxtral usa 32 cabezas para las consultas y solo 8 para las claves y los valores, una variante que reduce la memoria necesaria para guardar estos últimos. Como la atención es indiferente al orden, la posición de cada elemento se inyecta aparte, mediante codificaciones posicionales.

2.2.2. Codificador, decodificador y generación autorregresiva

Un Transformer de codificador y decodificador combina dos pilas de capas. Cada capa del codificador aplica autoatención sobre la entrada y después una red de alimentación hacia delante a cada posición. El decodificador genera la salida token a token: en cada paso recibe los tokens ya producidos, aplica autoatención sobre ellos y atención cruzada sobre la salida del codificador, en la que las consultas proceden del decodificador y las claves y valores, del codificador. Un token es la unidad mínima de texto del modelo, normalmente un fragmento de palabra. La figura 2 resume el esquema aplicado al reconocimiento del habla.

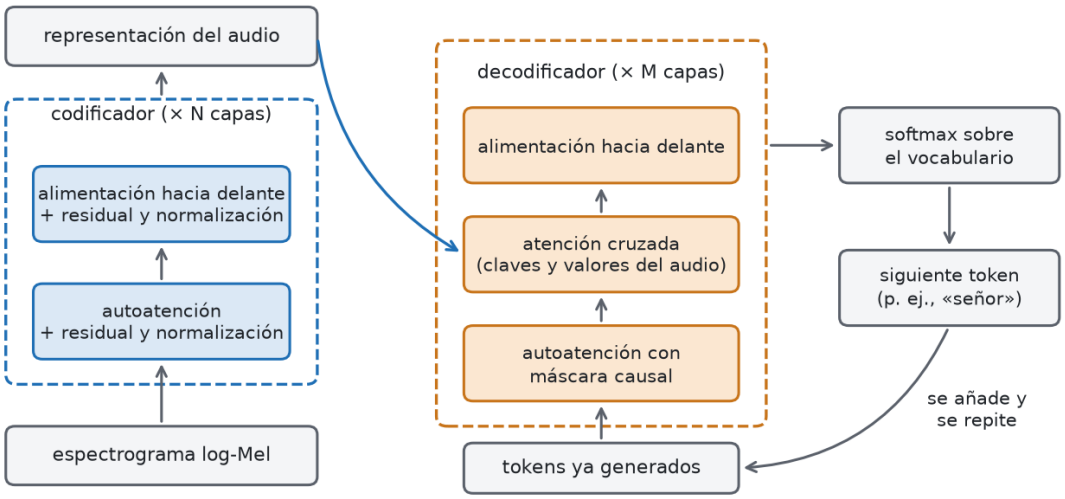


Figura 2. *Esquema de un Transformer de codificador y decodificador aplicado al reconocimiento del habla. Cada subcapa va acompañada de una conexión residual y una normalización, que en el decodificador se omiten por claridad. Elaboración propia a partir de Vaswani et al. (2017).*

Para que el decodificador no pueda ver los tokens futuros durante el entrenamiento se aplica una máscara causal, que anula los pesos de atención de cada posición hacia las posteriores. La misma máscara aplicada al codificador lo convierte en causal: cada vector depende solo del audio pasado. Es la propiedad que distingue al codificador de Voxtral Realtime del de Whisper.

La generación es autorregresiva: el modelo produce una distribución de probabilidad sobre el vocabulario, elige un token, lo añade a la secuencia y repite. Elegir siempre el más probable es la decodificación voraz; explorar varias hipótesis a la vez es la búsqueda en haz (Jurafsky y Martin 2026). Cada token elegido es, en rigor, una decisión de clasificación entre decenas de miles de clases. Para no recalcular en cada paso las claves y los valores de todos los tokens anteriores, se guardan en memoria en la llamada caché de claves y valores (caché KV). Su tamaño crece con la longitud de la secuencia, y es la memoria que gestiona vLLM y la que fija el techo de contexto de Qwen3-ASR en el capítulo 4.

2.2.3. Tokens y vocabulario

El vocabulario de un modelo de lenguaje no está hecho de palabras, sino de fragmentos obtenidos automáticamente a partir de un corpus: por construcción, las palabras frecuentes tienden a ocupar un único token y las raras, como un apellido poco común o un término técnico, se dividen en varios. Eso puede contribuir a que los nombres propios concentren tantos errores en el capítulo 4: el modelo tiene que acertar varias piezas seguidas, ninguna de ellas frecuente. Qwen3 usa una codificación por pares de bytes con un vocabulario de 151.669 tokens (Yang et al. 2025); la configuración publicada de Voxtral Realtime declara un vocabulario de 131.072.

El token tiene además, en Voxtral Realtime, un significado temporal. Como el decodificador produce exactamente un token por cada trama de 80 ms, genera 12,5 tokens por segundo hable o no el orador, y la mayoría son tokens de espera: a la velocidad de habla de Europarl-ST, de unas 150 palabras por minuto (sección 3.5.2), llegan dos o tres palabras por segundo frente a doce pasos y medio. El coste del modelo depende, por tanto, de la duración del audio y no de la cantidad de texto, una propiedad que reaparece al explicar por qué su modo diferido no es más rápido que el streaming.

2.2.4. La arquitectura de los decodificadores actuales

El Transformer original rodea cada subcapa de atención y de alimentación hacia delante con una conexión residual, que suma la entrada de la subcapa a su salida, y con una normalización de capa, que reescala las activaciones para estabilizar el entrenamiento (Vaswani et al. 2017). Las conexiones residuales permiten apilar decenas de capas sin que el gradiente se desvanezca, y son la razón de que los decodificadores de los dos modelos evaluados tengan 26 y 28 capas sin dificultades de entrenamiento.

Los modelos de lenguaje actuales conservan ese esqueleto con algunas variantes que conviene reconocer, porque aparecen en la descripción de los dos modelos evaluados. Qwen3, la familia de la que procede el decodificador de Qwen3-ASR, usa atención con consultas agrupadas (varias cabezas de consulta comparten las mismas claves y valores, lo que reduce la caché KV), una función de activación SwiGLU en la red de alimentación hacia delante, codificaciones posicionales rotatorias (RoPE) y RMSNorm, una normalización que solo reescala por la media cuadrática de las activaciones, aplicada antes de cada subcapa (Yang et al. 2025). La configuración publicada de Voxtral Realtime muestra los mismos ingredientes: 32 cabezas de consulta frente a 8 de claves y valores, posiciones rotatorias y normalización RMS. Las posiciones rotatorias merecen una línea: en lugar de sumar un vector de posición a la entrada, giran las consultas y las claves un ángulo proporcional a su posición, de modo que el producto

escalar entre ambas depende de su distancia relativa y no de su posición absoluta. La propiedad encaja con la atención por ventana deslizante de Voxtral Realtime, en la que lo que importa es la distancia entre posiciones dentro de la ventana y no el instante absoluto de la intervención. Sobre la RMSNorm construye Voxtral Realtime su normalización adaptativa, que inyecta el retardo de transcripción en cada capa (sección 3.2.2).

2.3. Modelos fundacionales del habla

El término modelo fundacional designa a los modelos entrenados con grandes volúmenes de datos amplios y adaptables a un gran número de tareas posteriores (Bommasani et al. 2021). Sus proponentes eligieron la palabra para subrayar a la vez su carácter central y su carácter incompleto: un modelo fundacional es una base sobre la que se construye, y sus defectos los heredan todos los sistemas que se apoyan en él.

En el habla, dos estrategias de entrenamiento a gran escala abrieron el camino. La primera es la autosupervisión. wav2vec 2.0 aprende representaciones del habla a partir de audio sin transcribir: enmascara partes de la representación latente de la entrada y resuelve una tarea contrastiva, distinguir la representación verdadera de un fragmento enmascarado de otras que actúan como distractores (Baevski et al. 2020). Con un preentrenamiento sobre 53.000 horas de audio sin etiquetar y solo diez minutos de datos transcritos, sus autores obtienen un 4,8 y un 8,2 % de WER en los conjuntos limpio y difícil de LibriSpeech. La autosupervisión mostró que la mayor parte de lo que un reconocedor necesita saber se puede aprender sin transcripciones.

La segunda es la supervisión débil, y su exponente es Whisper (Radford et al. 2023). En lugar de transcripciones cuidadas, Whisper se entrenó con 680.000 horas de audio de internet acompañado de textos de calidad variable, en muchos idiomas y con varias tareas a la vez: transcripción, traducción al inglés e identificación del idioma. El resultado es un modelo de codificador y decodificador que generaliza bien a conjuntos de evaluación que nunca ha visto, sin ajuste alguno, y se acerca a la precisión y a la robustez humanas. Whisper fijó el patrón que siguen los modelos actuales: un codificador de audio que alimenta a un decodificador de lenguaje, entrenados con datos masivos, multilingües y multitarea.

Voxtral y Qwen3-ASR son herederos de ese patrón con una diferencia: su decodificador no se entrena desde cero, sino que parte de un modelo de lenguaje de texto ya entrenado, Ministral 3B en un caso y Qwen3-1.7B en el otro (Mistral AI 2026; Shi et al. 2026). El reconocedor hereda así el conocimiento del idioma que el modelo de lenguaje adquirió con texto, y la tarea del entrenamiento con audio se reduce a enseñarle a escuchar.

La primera versión de Voxtral ilustra bien la receta (Liu et al. 2025). Un codificador de audio derivado del de Whisper large-v3 procesa la grabación en fragmentos de 30 segundos; un adaptador agrupa sus salidas de cuatro en cuatro, hasta una cadencia de 12,5 vectores por segundo; y un decodificador de lenguaje, Ministral 3B en la versión Mini, recibe esos vectores como si fueran tokens y genera la respuesta. Con una ventana de contexto de 32K tokens, la cadencia de 12,5 vectores por segundo permite procesar hasta 40 minutos de audio en una sola llamada. El modelo no solo transcribe: responde preguntas sobre el audio y mantiene conversaciones, porque su decodificador conserva lo que sabía hacer con texto. Para el tiempo real, sin embargo, la receta tiene un

inconveniente estructural: el codificador de Whisper atiende en ambas direcciones dentro de cada fragmento de 30 segundos y necesita el fragmento completo para empezar. Voxtral Realtime, descrito en el capítulo 3, sustituye precisamente esa pieza por un codificador causal entrenado desde cero.

2.4. Reconocimiento en tiempo real

2.4.1. Causalidad y procesamiento incremental

Un sistema de transcripción en tiempo real, o en streaming, emite texto mientras el audio llega, sin esperar a que termine. Eso impone dos condiciones. La primera es de información: el sistema solo puede usar el audio recibido hasta el momento, más, como mucho, un pequeño margen de audio futuro que decida esperar. La segunda es de cómputo: el trabajo necesario para incorporar cada nuevo fragmento debe caber en lo que dura ese fragmento, o el sistema se irá retrasando sin remedio.

La autoatención del Transformer choca con ambas. Si cada posición atiende a toda la secuencia, cada nuevo fragmento modifica la representación de todos los anteriores y obliga a recalcularlos. La solución general combina causalidad y memoria. Emformer, un modelo acústico para streaming, procesa el audio en bloques, resume el contexto lejano en un banco de memoria y guarda en caché las claves y los valores del contexto inmediato para no recalcularlos (Shi et al. 2021). Un trabajo posterior adapta un codificador Conformer limitando el contexto pasado y futuro de cada capa y guardando sus activaciones en caché, de modo que un codificador no autorregresivo funcione de forma incremental; muestra además que entrenar con varias latencias a la vez permite atender varias con un solo modelo (Noroozi et al. 2023). La idea común es mantener un estado: lo ya calculado no se repite, y el coste de cada paso no depende de cuánto audio se ha procesado antes.

2.4.2. Adaptar modelos diferidos o diseñar modelos nativos

Hay dos maneras de obtener un reconocedor en tiempo real a partir de un modelo fundacional. La primera es adaptar uno diseñado para transcribir grabaciones completas. Whisper-Streaming, por ejemplo, alimenta a Whisper con un búfer de audio creciente, lo vuelve a transcribir cada vez que llega audio nuevo y solo confirma las palabras en las que coinciden dos transcripciones consecutivas; obtiene alta calidad con una latencia de 3,3 segundos sobre habla larga sin segmentar (Macháček, Dabre y Bojar 2023). La estrategia funciona con cualquier modelo diferido, pero paga dos precios: la latencia, porque hay que esperar a que las hipótesis se estabilicen, y el cómputo, porque el mismo audio se procesa muchas veces.

La segunda es diseñar el modelo para el streaming desde el entrenamiento. El modelado de flujos retardados trata el audio y el texto como dos flujos ya alineados en el tiempo y procesados por un único decodificador; para el reconocimiento, el flujo de texto se desplaza un retardo fijo respecto del de audio, de manera que el modelo aprende a escribir cada palabra un tiempo después de haberla oído (Zeghidour et al. 2025). La alineación se resuelve en un paso de preprocesado y no en la inferencia, y la latencia queda fijada por construcción.

Qwen3-ASR sigue en su modo streaming una variante de la primera vía, con reprocesado del audio acumulado; Voxtral Realtime, la segunda.

2.4.3. Latencia algorítmica, computacional y total

Tiempo real no significa instantáneo. Entre que se pronuncia una palabra y aparece en pantalla transcurren dos intervalos de naturaleza distinta, representados en la figura 3.

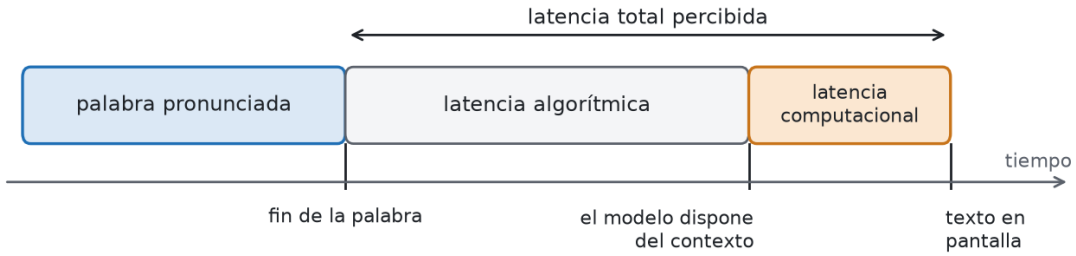


Figura 3. *Componentes de la latencia percibida al transcribir una palabra en tiempo real. Elaboración propia.*

La latencia algorítmica es la espera que impone el diseño del sistema, porque necesita oír algo de audio posterior antes de decidir una palabra o porque acumula el audio en bloques antes de procesarlo. No depende del equipo: en Voxtral Realtime es el retardo de transcripción configurado; en Qwen3-ASR, el tamaño del bloque. La latencia computacional es el tiempo que tarda el equipo en procesar cada paso una vez dispone del audio, y esa sí depende del hardware, del sistema operativo y de la implementación. La latencia total que percibe el usuario es, en primera aproximación, la suma de las dos.

Entre las dos latencias y la calidad hay un compromiso que la literatura documenta con claridad. Emformer, por ejemplo, obtiene en LibriSpeech un WER del 2,50 % en el conjunto limpio y del 5,62 % en el difícil con una latencia media de 960 ms, y del 3,01 y el 7,09 % cuando la latencia baja a 80 ms (Shi et al. 2021). Reducir la espera doce veces cuesta, en ese caso, entre medio punto y punto y medio de error. Voxtral Realtime muestra la misma forma de curva en FLEURS, como se verá en la sección 3.2.3. Ese compromiso es el que el usuario de un sistema en tiempo real elige, a sabiendas o no, cada vez que fija un retardo.

La distinción entre latencia algorítmica y computacional tiene, además, una consecuencia metodológica que este trabajo toma como regla. Un sistema que espera más dispone de más contexto y acierta más, de modo que comparar dos sistemas con latencias algorítmicas distintas no dice cuál reconoce mejor, sino cuál espera más. La comparación honesta se hace a igual latencia algorítmica, y se completa midiendo la computacional.

2.4.4. Hipótesis definitivas y revisables

Un sistema en tiempo real no solo decide cuándo escribir una palabra, sino también si podrá corregirla después. Hay dos políticas. En la primera, cada token emitido es definitivo: el sistema espera lo necesario para estar razonablemente seguro y no vuelve atrás. Es la de los modelos de flujos retardados como Voxtral Realtime, que escriben cada palabra cuando ha transcurrido el retardo y no la revisan. En la segunda, la hipótesis es revisable: el sistema muestra pronto una transcripción provisional y la corrige a medida que llega más audio. Whisper-Streaming, por ejemplo, solo confirma el prefijo en el que coinciden dos transcripciones sucesivas y mantiene el resto abierto

(Macháček, Dabre y Bojar 2023); Qwen3-ASR deja sin fijar los últimos tokens de cada hipótesis para poder reescribirlos en el paso siguiente (sección 3.3.2).

Las dos políticas reparten de forma distinta el mismo compromiso. La hipótesis revisable muestra texto antes, pero ese texto puede cambiar ante los ojos del lector, algo que resulta molesto en un subtítulo y problemático para cualquier proceso que consuma la transcripción, como un traductor automático, que tendría que rehacer su trabajo con cada corrección. La hipótesis definitiva es estable, pero solo es tan buena como el contexto que el retardo le permite ver. En la latencia de un sistema con hipótesis revisables hay, por tanto, una ambigüedad que conviene tener presente al comparar: no es lo mismo el momento en que aparece una palabra que el momento en que deja de cambiar. Las medidas de este trabajo usan para Qwen3-ASR el tamaño del bloque como latencia algorítmica, que corresponde al primero de esos momentos y es, por tanto, una estimación favorable a ese modelo.

2.5. Métricas

2.5.1. Tasa de error de palabra y normalización

La métrica estándar del reconocimiento es la tasa de error de palabra (WER, por *word error rate*). Se alinean la transcripción del sistema, o hipótesis, y la referencia de manera que el número de operaciones de edición para convertir una en otra sea mínimo, y se cuentan las sustituciones S, las eliminaciones D (palabras de la referencia que faltan en la hipótesis) y las inserciones I (palabras de la hipótesis que no están en la referencia). Con N palabras en la referencia (Jurafsky y Martin 2026):

$$\mathrm{WER} = \frac{S + D + I}{N} \times 100$$

La misma fórmula aplicada a caracteres en lugar de palabras da la tasa de error de caracteres (CER), que es la que usaron los autores de Europarl-ST para filtrar el corpus (sección 3.5.1). Al contar las inserciones, el WER no está acotado superiormente: un sistema que repite una frase sin fin puede superar el 100 %, como sucede en el capítulo 4. La tabla 3 muestra un ejemplo real, tomado de la salida de Voxtral sobre la intervención de la figura 1.

Tabla 3. *Alineamiento de dos fragmentos de una transcripción de Voxtral con su referencia. Sobre las 11 palabras de referencia de los dos fragmentos hay una sustitución y dos inserciones: un WER del 27,27 %.*

Referencia	Hipótesis	Operación
señor	señor	Acierto
presidente	presidente	Acierto
	gracias	Inserción (I)
en	en	Acierto
un	un	Acierto
tema	tema	Acierto
…	…	
como	como	Acierto
el	el	Acierto
fallido	fallito	Sustitución (S)
	de	Inserción (I)
atentado	atentado	Acierto
de	de	Acierto
detroit	detroit	Acierto

Antes de comparar, hipótesis y referencia se normalizan, porque la referencia de Europarl-ST y la salida de un modelo moderno no siguen las mismas convenciones de puntuación ni de mayúsculas, y esas diferencias no son errores de reconocimiento. En este trabajo se usa el normalizador básico que acompaña al código de Whisper (Radford et al. 2023):1 pasa el texto a minúsculas, elimina lo que aparece entre corchetes o paréntesis, sustituye cada signo de puntuación o símbolo por un espacio y reduce los espacios repetidos a uno. Conserva las tildes. El apóstrofo también es un signo de puntuación, y por eso la forma contraída inglesa *don't* queda como «don t», dos palabras que no coinciden con las de *do not*; el análisis del inglés del capítulo 4 vuelve sobre ello. El cálculo se hace con la biblioteca jiwer,2 sobre el conjunto completo de intervenciones: se suman los errores de todas y se dividen entre el total de palabras de referencia. Así, una intervención larga pesa más que una corta, a diferencia de lo que ocurriría con la media de los WER individuales.

2.5.2. Factor de tiempo real

El factor de tiempo real (RTF, por *real-time factor*) es el cociente entre el tiempo de proceso y la duración del audio procesado:

$$\mathrm{RTF} = \frac{t_{\mathrm{proceso}}}{t_{\mathrm{audio}}}$$

Un RTF de 0,5 significa que una hora de audio se procesa en media hora. Es una medida de capacidad, útil para dimensionar el hardware de un servicio de transcripción de archivos. En tiempo real es necesario que el RTF sea menor que uno, pero no es suficiente: un sistema puede procesar en promedio más deprisa de lo que llega el audio y, aun así, hacer esperar al usuario segundos por cada palabra, si acumula el audio en bloques grandes o si su coste se concentra al final.

2.5.3. Uso del paso y crecimiento

Para caracterizar el comportamiento en tiempo real con más precisión que el RTF, este trabajo usa dos indicadores propios, calculados a partir del tiempo de cómputo de cada llamada al modelo. Un paso es cada una de esas llamadas: en Voxtral Realtime incorpora 80 ms de audio nuevo; en Qwen3-ASR, un bloque del tamaño configurado.

El uso del paso es el porcentaje que la latencia computacional media representa sobre la duración del audio nuevo de cada paso. Por debajo del 100 % el sistema termina un paso antes de que esté disponible el siguiente; por encima, acumula retraso.

El crecimiento es el cociente entre la latencia computacional media del último tercio de los pasos de una intervención y la del primer tercio, promediadas sobre todas las intervenciones evaluadas. Vale aproximadamente uno cuando el coste de cada paso es constante y supera la unidad cuando el sistema se ralentiza a medida que avanza la intervención.

Ninguno de los dos basta por separado. Un uso del paso bajo con un crecimiento alto anuncia un sistema que llega holgado al principio y se queda atrás al final; un uso alto con crecimiento uno, uno que va justo pero no empeora. Solo la combinación permite

1 Repositorio de Whisper, con el normalizador de texto. Disponible en: https://github.com/openai/whisper (consulta: 19-10-2026).
2 jiwer. Disponible en: https://github.com/jitsi/jiwer (consulta: 19-10-2026).

afirmar que un sistema sostiene el tiempo real en intervenciones de cualquier duración, y es el criterio que se aplica en el capítulo 4.

3. Modelos fundacionales de ASR y conjunto de datos Europarl-ST

Un experimento vale lo que valen sus piezas. Este capítulo presenta las tres que sostienen todo lo que viene después: los dos modelos evaluados, Voxtral Mini 4B Realtime y Qwen3-ASR 1.7B, y el corpus sobre el que se miden, Europarl-ST. Ninguna de las tres se describe aquí por completo, porque sus artículos lo hacen mejor y con más autoridad; se describe lo que hace falta para entender por qué los dos modelos, puestos ante la misma intervención parlamentaria, se comportan como se comportan.

3.1. Criterios de selección

La propuesta del trabajo partía de Voxtral y dejaba abierta la puerta a un segundo modelo con el que contrastarlo. Elegir ese segundo modelo exigía cuatro condiciones, y ninguna era negociable. La primera, pesos abiertos: sin acceso al modelo no hay forma de medir su latencia, sino solo la de una red y un servicio ajenos. La segunda, cobertura de los cuatro idiomas del corpus (alemán, castellano, francés e inglés). La tercera, un modo de funcionamiento en tiempo real ofrecido por los propios autores, no improvisado sobre un modelo pensado para transcribir archivos completos. La cuarta, más prosaica y quizá la más restrictiva, que cupiera en la tarjeta disponible, una NVIDIA RTX 3060 con 12 GB de memoria.º

Qwen3-ASR 1.7B cumple las cuatro. Se publicó en enero de 2026 bajo licencia Apache 2.0 (Shi et al. 2026), reconoce 30 idiomas (entre ellos los cuatro necesarios) y 22 dialectos del chino, y ofrece un modo de inferencia en streaming. Voxtral Mini 4B Realtime, publicado un mes después con la misma licencia (Mistral AI 2026), cubre 13 idiomas, también los cuatro del corpus. Los dos son, por tanto, comparables en lo formal. Lo interesante, como se verá, es que no lo son en lo esencial: resuelven el tiempo real con filosofías opuestas.

3.2. Voxtral Mini 4B Realtime

Voxtral Realtime es la variante de transmisión continua de la familia Voxtral, cuyos primeros modelos estaban pensados para transcribir y comprender audio completo (Liu et al. 2025). Sus autores lo presentan como un modelo nativamente diseñado para el streaming, entrenado de extremo a extremo para emitir texto mientras el audio llega, y no como un modelo diferido al que se trocea la entrada (Mistral AI 2026). La diferencia no es de matiz: un modelo entrenado viendo la grabación completa aprende a apoyarse en lo que todavía no ha sonado, y cuando se le niega ese futuro su calidad se resiente tanto más cuanto más corta es la espera que se le concede.

3.2.1. Arquitectura

El modelo se compone de tres bloques encadenados, cuyas dimensiones recoge la tabla 4. Un codificador de audio convierte la señal en una secuencia de vectores; un adaptador reduce la frecuencia de esa secuencia; un decodificador de lenguaje genera el texto token a token.

Tabla 4. *Configuración de Voxtral Realtime. Fuente: Mistral AI (2026).*

Componente	Capas	Dimensión	Cabezas	Ventana deslizante	Parámetros
Codificador de audio	32	1280	32	750 tramas	970 M
Adaptador	1	1280 × 4 → 3072			25 M
Decodificador de lenguaje	26	3072	32 (8 para claves y valores)	8192 tokens	3,4 B
Total					4,4 B

El codificador recibe un espectrograma log-Mel de 128 bandas calculado con saltos de 10 ms sobre audio a 16 kHz. Una primera etapa convolucional causal reduce la frecuencia a la mitad y entrega un vector cada 20 ms; sobre ella se apilan 32 capas de autoatención, también causales, que solo miran hacia atrás. Esa causalidad es la decisión de diseño de la que se deriva casi todo lo demás. El codificador de Whisper, el modelo de referencia en transcripción diferida (Radford et al. 2023), atiende en ambas direcciones; el de Voxtral Realtime se entrenó desde cero para no hacerlo nunca, y sus autores detallan las diferencias entre ambos (Mistral AI 2026). La atención se limita además a una ventana deslizante de 750 tramas, es decir, 15 segundos de audio, de modo que la memoria necesaria no crece aunque la intervención dure una hora.

El adaptador agrupa cuatro vectores consecutivos y los proyecta a la dimensión del decodificador. La secuencia resultante avanza a 12,5 tramas por segundo: cada trama representa 80 ms de audio. Esa cifra, 80 ms, es la unidad de tiempo del modelo y reaparecerá en todas las medidas de latencia del capítulo 4.

El decodificador es un Transformer de solo decodificador que parte de los pesos de Ministral 3B y conserva también una ventana deslizante, en su caso de 8192 tokens. En cada paso suma el vector de audio de la trama actual con el del último token generado y produce exactamente un token. Ni más ni menos. Tampoco aquí la memoria crece con la duración, y por eso los autores pueden afirmar que el modelo admite flujos de longitud arbitraria con memoria acotada.

La ficha oficial del modelo indica que su despliegue con un servidor de inferencia requiere una GPU de al menos 16 GB.[3] En este trabajo se cargó con la biblioteca transformers en precisión bfloat16, sin servidor, y los pesos ocuparon 8,25 GiB de la RTX 3060. Cabe, pero sin holgura para un segundo modelo.

3.2.2. El retardo como condición del modelo

¿Cómo sabe un modelo causal cuándo debe escribir una palabra? No lo sabe por sí mismo: se le enseña. Durante el entrenamiento, cada trama de 80 ms tiene asignado un token objetivo. Mientras no hay ninguna palabra completa que transcribir, el objetivo es un token de espera, [P]. Cuando una palabra ya se ha oído entera y ha transcurrido además un retardo τ, el objetivo pasa a ser un token de frontera, [W], seguido de los fragmentos de la palabra, uno por trama. La figura 4 muestra el mecanismo con un retardo de 240 ms. El modelo aprende así, sin alineamientos externos ni detección de voz, cuándo esperar y cuándo hablar.

3 Ficha de Voxtral Mini 4B Realtime en Hugging Face. Disponible en: https://huggingface.co/mistralai/Voxtral-Mini-4B-Realtime-2602 (consulta: 19-10-2026).

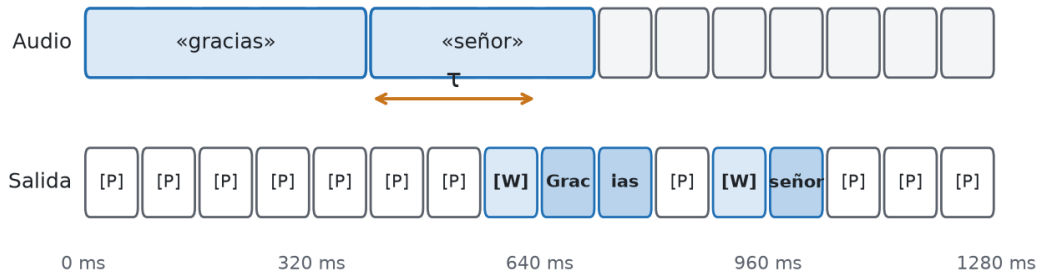


Figura 4. *Esquema de emisión de Voxtral Realtime con un retardo τ de 240 ms (tres tramas). Cada casilla es una trama de 80 ms. La segmentación de las palabras en tokens es ilustrativa. Elaboración propia a partir de Mistral AI (2026).*

Este esquema hereda la idea del modelado de flujos retardados, que alinea un flujo de audio y uno de texto desplazados en el tiempo y los procesa con un único decodificador (Zeghidour et al. 2025). Lo que Voxtral Realtime añade es la posibilidad de elegir τ en el momento de usarlo. Durante el entrenamiento, el retardo se sortea entre 80 y 2400 ms en pasos de 80 ms, y su valor se inyecta en cada bloque del decodificador mediante una normalización adaptativa (Ada RMS-Norm): un pequeño perceptrón transforma τ en un vector que modula la rama de alimentación hacia delante de cada capa. Un solo conjunto de pesos sirve entonces para cualquier retardo entrenado.

Que sirva no significa que sirva igual de bien en todos. La ficha del modelo limita los valores admitidos a los múltiplos de 80 ms entre 80 y 1200 ms, más el valor aislado de 2400 ms, y recomienda 480 ms, que es también el valor por defecto. En la implementación de transformers, el retardo se fija en la configuración del procesador (transcription_delay_ms), y el audio se entrega en pasos de 80 ms que el procesador convierte en rasgos sobre una ventana de 105 ms, solapada con la anterior. El modelo no admite que se le indique el idioma: lo deduce del propio audio.

3.2.3. Resultados publicados

El artículo de Voxtral Realtime mide la calidad frente al retardo en varios conjuntos, y el más cercano a este trabajo es FLEURS, que reúne frases escritas de antemano grabadas por tres hablantes nativos en cada uno de sus 102 idiomas (Conneau et al. 2023). La tabla 5 extrae los cuatro idiomas que aquí interesan.

Tabla 5. *WER (%) en FLEURS de Voxtral Realtime según el retardo, frente a Whisper en diferido. Fuente: Mistral AI (2026), tabla 7.*

Modelo	Retardo	Alemán	Castellano	Francés	Inglés
Whisper (diferido)		5,46	2,81	5,55	4,00
Voxtral Realtime	240 ms	8,15	4,59	8,00	5,91
Voxtral Realtime	480 ms	6,19	3,31	6,42	4,90
Voxtral Realtime	960 ms	4,87	2,98	5,68	4,34
Voxtral Realtime	2400 ms	4,15	2,71	5,23	4,05

Dos lecturas saltan a la vista. A 480 ms, el modelo en tiempo real queda a menos de un punto de Whisper en los cuatro idiomas; a 960 ms ya lo mejora en alemán y se queda a menos de cuatro décimas en los otros tres. Y la mejora con el retardo es monótona, sin saltos: más espera, menos error. ¿Garantizan estas cifras un comportamiento parecido ante el Parlamento Europeo? No lo garantizan. FLEURS se compone de frases leídas,

limpias y breves; Europarl-ST, de intervenciones reales grabadas en el hemiciclo, con la duración, la acústica y la terminología propias de un debate. El capítulo 4 mide la distancia entre ambos mundos, y no es pequeña.

3.3. Qwen3-ASR 1.7B

3.3.1. Arquitectura y entrenamiento

Qwen3-ASR se construye sobre un modelo multimodal más general, Qwen3-Omni, y combina un codificador de audio propio con un decodificador de lenguaje de la familia Qwen3 (Shi et al. 2026; Yang et al. 2025). En la versión de 1,7 B, el decodificador es Qwen3-1.7B y el codificador, llamado AuT, tiene 300 M de parámetros y una dimensión oculta de 1024. AuT se preentrena por separado como un sistema de reconocimiento de codificador y decodificador con atención; reduce por ocho la frecuencia de un banco de filtros de 128 dimensiones y entrega 12,5 vectores por segundo, la misma cadencia de 80 ms que Voxtral.

El parecido termina ahí. El codificador de Qwen3-ASR no es causal: usa una ventana de atención dinámica de entre 1 y 8 segundos, y es esa ventana variable la que, según sus autores, le permite servir tanto para inferencia en streaming con fragmentos cortos como para inferencia diferida con audio largo. El modelo identifica el idioma por sí mismo, pero también acepta que se le fije, posibilidad que este trabajo aprovecha en todos los experimentos con Qwen3-ASR. Admite audios de hasta 20 minutos en una sola llamada.

Su entrenamiento tiene cuatro etapas (Shi et al. 2026). En la primera, el codificador AuT se preentrena como reconocedor de codificador y decodificador con atención sobre unos 40 millones de horas de datos de reconocimiento etiquetados automáticamente, en su mayoría en chino e inglés. En la segunda, el modelo completo hereda el preentrenamiento multimodal de Qwen3-Omni, con 3 billones de tokens de audio, imagen y texto. La tercera es un ajuste supervisado con un conjunto multilingüe mucho menor, que incluye datos sin habla, datos de refuerzo para el streaming y datos de sesgo contextual; en ella se entrena al modelo para que solo transcriba y no obedezca instrucciones en lenguaje natural, con el fin de evitar inyecciones de instrucciones. La cuarta es una etapa de aprendizaje por refuerzo, con unos 50.000 enunciados, que según el informe resulta esencial para la robustez ante el ruido, la estabilidad de la transcripción y los casos difíciles. La estabilidad de la transcripción, como se verá en el capítulo 4, no impidió un bucle de repetición en una intervención francesa.

3.3.2. Streaming por reprocesado

La ficha del modelo advierte que el streaming solo está disponible con el motor de inferencia vLLM, y que en ese modo no se admiten lotes ni marcas de tiempo.4 vLLM es un servidor de modelos de lenguaje que gestiona la caché de claves y valores por páginas para aprovechar mejor la memoria de la GPU (Kwon et al. 2023).5

4 Ficha de Qwen3-ASR 1.7B en Hugging Face. Disponible en: https://huggingface.co/Qwen/Qwen3-ASR-1.7B (consulta: 19-10-2026).
5 Documentación de vLLM. Disponible en: https://docs.vllm.ai/ (consulta: 19-10-2026).

El mecanismo de streaming se lee directamente en el código del paquete oficial.6 El audio se acumula hasta completar un bloque, de 2 segundos por defecto. Cada vez que se completa uno, el bloque se añade a todo el audio recibido hasta ese momento y el conjunto entero se vuelve a pasar por el modelo. Para no perder la coherencia del texto, la transcripción anterior se usa como prefijo de la nueva, salvo sus últimos cinco tokens, que se dejan abiertos a corrección; durante los dos primeros bloques no se usa prefijo alguno. El informe técnico evalúa este modo con bloques de 2 segundos, una reversión de cinco tokens y cuatro bloques sin fijar, y lo compara con el diferido en tres conjuntos públicos (Shi et al. 2026). La tabla 6 recoge sus cifras.

Tabla 6. *Tasa de error (%) de Qwen3-ASR 1.7B en diferido y en streaming según su informe técnico. Fuente: Shi et al. (2026), tabla 8.*

Modo	LibriSpeech limpio	LibriSpeech difícil	FLEURS inglés	FLEURS chino	Media
Diferido	1,63	3,38	3,35	2,41	2,69
Streaming, bloques de 2 s	1,95	4,51	4,02	2,84	3,33

La pérdida media es de 0,64 puntos, y en ningún conjunto supera 1,2. Son cifras sobre habla leída y con bloques de 2 segundos, es decir, con una latencia algorítmica cuatro veces mayor que el valor por defecto de Voxtral.

La estrategia es sencilla y funciona. Tiene, sin embargo, un precio que el informe no discute y que la figura 5 hace visible. Si cada paso reprocesa todo lo anterior, el audio que el modelo analiza en cada paso crece linealmente con la duración de la intervención, y el acumulado lo hace de forma cuadrática. Con bloques de 0,48 s, dos minutos de intervención obligan a procesar unas 15.000 s de audio en total, más de cuatro horas; con bloques de 2 s, unas 3660 s. Voxtral, que avanza 80 ms por paso sobre una ventana de 105 ms y conserva el resto en sus cachés, procesa unos 158 s.

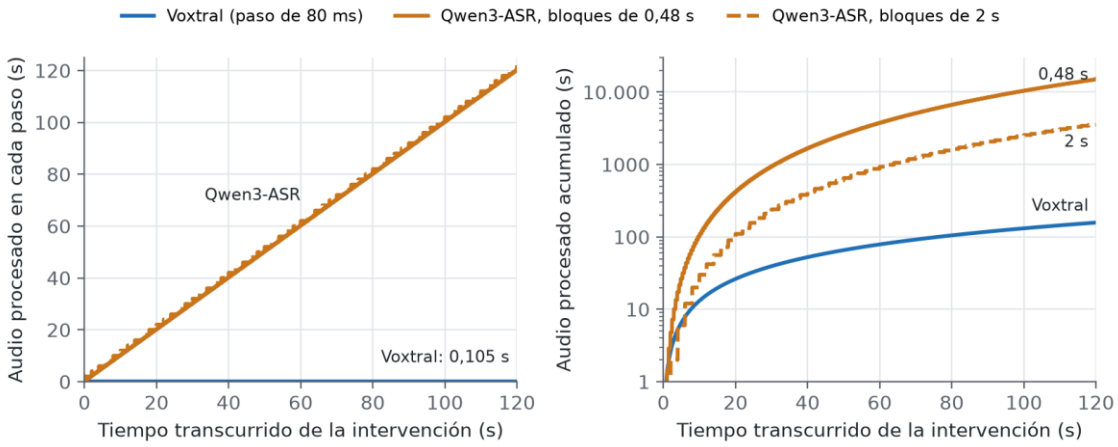


Figura 5. *Audio que procesa cada modelo en cada paso (izquierda) y acumulado (derecha, escala logarítmica) a lo largo de una intervención de 120 s. Las curvas se calculan a partir del mecanismo de cada modelo, no de mediciones. Elaboración propia.*

De este mecanismo se siguen dos consecuencias que el capítulo 4 confirmará con medidas. La primera es que el coste de cada paso no es constante: crece dentro de la

6 Repositorio de Qwen3-ASR. Disponible en: https://github.com/QwenLM/Qwen3-ASR (consulta: 19-10-2026).

propia intervención, y tanto más deprisa cuanto menor es el bloque. La segunda es que la secuencia que recibe el modelo también crece, porque incluye el audio acumulado y el texto ya generado, de modo que tarde o temprano choca con la longitud máxima que el motor puede reservar en memoria. Hay, pues, un techo de contexto. Bajar la latencia, que exige bloques más pequeños, acerca ese techo.

3.4. Dos maneras de entender el tiempo real

La tabla 7 reúne las diferencias que importan para este trabajo.

Tabla 7. *Comparación de los dos modelos evaluados. Elaboración propia a partir de Mistral AI (2026), Shi et al. (2026) y de sus fichas oficiales.*

Característica	Voxtral Mini 4B Realtime	Qwen3-ASR 1.7B
Parámetros	4,4 B	1,7 B de decodificador y 300 M de codificador
Cadencia del codificador tras la reducción	12,5 Hz (80 ms)	12,5 Hz (80 ms)
Codificador	Causal, ventana de 15 s	Ventana de atención dinámica de 1 a 8 s
Parámetro de latencia	Retardo transcription_delay_ms, múltiplos de 80 ms	Tamaño de bloque chunk_size_sec
Valor por defecto	480 ms	2 s
Coste por paso en streaming	Constante: solo el audio nuevo	Creciente: todo el audio acumulado
Idioma	Detección automática, no se puede fijar	Detección automática o fijado por el usuario
Inferencia en streaming	transformers o vLLM	Solo vLLM
Licencia	Apache 2.0	Apache 2.0

Una fila merece detenerse. Con sus valores por defecto, Voxtral espera 480 ms y Qwen3-ASR acumula 2 s antes de emitir, de modo que compararlos tal cual vienen sería comparar sistemas que no prometen lo mismo. El de mayor espera dispone de más contexto y, previsiblemente, acierta más. Por eso el capítulo 4 los compara a igual latencia algorítmica, y por eso esa comparación solo es posible en un punto: 480 ms, el valor nativo de Voxtral y el bloque más pequeño que se ensayó con Qwen3-ASR.

3.5. El corpus Europarl-ST

3.5.1. Origen y construcción

Europarl-ST es un corpus multilingüe para traducción del habla construido por el grupo MLLP del instituto VRAIN de la Universitat Politècnica de València a partir de los vídeos públicos de los debates del Parlamento Europeo entre 2008 y 2012 (Iranzo-Sánchez et al. 2020).7 Su unidad básica es la intervención: lo que dice un único orador en su turno. Cada intervención va acompañada de su transcripción oficial y, para la traducción, de su versión en otras lenguas. Se distribuye con licencia Creative Commons de reconocimiento no comercial (CC BY-NC 4.0).

Su construcción tiene interés para quien lo usa como banco de pruebas, porque explica tanto su calidad como algunas de sus asperezas. Las marcas de tiempo originales de los debates eran a menudo inexactas y arrastraban fragmentos de la intervención anterior o de la siguiente. Para corregirlo, los autores aplicaron diarización de locutores a cada

7 Página del corpus Europarl-ST. Disponible en: https://www.mllp.upv.es/europarl-st/ (consulta: 19-10-2026).

intervención, recortaron el tramo continuo más largo del mismo orador y lo alinearon de forma forzada con su transcripción. De ese recorte procede el nombre de los ficheros de audio del corpus, audio_clip_diarization.m4a. Después transcribieron automáticamente todas las intervenciones y descartaron aquellas cuya tasa de error de caracteres frente a la transcripción oficial superaba un umbral (15 % en alemán, castellano y francés; 20 % en inglés), señal de que la transcripción no era literal o de que el recorte había fallado. Entre ambos filtros se eliminó entre un 60 y un 80 % de los datos originales.

El filtrado garantiza que lo que queda es razonablemente limpio. No garantiza, en cambio, que el audio contenga solo lo transcrito. El análisis de errores del capítulo 4 encontró intervenciones que terminan con la respuesta de la Presidencia, presente en el audio y ausente de la referencia, lo que penaliza por igual a cualquier sistema que la transcriba.

3.5.2. La partición de test empleada

El corpus se organiza por pares de idiomas de origen y destino, y cada par tiene sus particiones de entrenamiento, desarrollo y test. Para evaluar reconocimiento solo importa el idioma de origen, así que se tomó, para cada uno de los cuatro idiomas, la partición de test de un par: castellano a inglés, inglés a castellano, francés a inglés y alemán a inglés. La tabla 8 resume su tamaño.

Tabla 8. *Partición de test usada en cada idioma y WER de los sistemas de referencia del artículo original. Fuente de la última columna: Iranzo-Sánchez et al. (2020), tabla 4.*

Idioma	Par	Versión del listado	Intervenciones	Horas	Palabras de referencia	WER del artículo
Castellano	es → en	v2	210	5,42	50.720	15,0 %
Inglés	en → es	v2	127	3,13	28.320	17,2 %
Francés	fr → en	v1	166	4,80	47.109	24,3 %
Alemán	de → en	v1	226	6,20	52.127	19,8 %
Total			729	19,56	178.276	

Los listados de castellano e inglés llevan en su nombre la etiqueta de una segunda versión del corpus; los de francés y alemán no llevan ninguna y se han tratado como de la primera. El matiz importa porque la línea base del artículo, publicada en 2020, corresponde a la primera versión, y en los dos idiomas con listado de la segunda la comparación con ella debe tomarse como orientativa.

Más allá del tamaño, interesan dos rasgos del test que condicionan los experimentos. El primero es la duración de las intervenciones. La tabla 9 la resume junto con la velocidad media de habla, calculada como palabras de referencia entre minutos de grabación.

Tabla 9. *Rasgos de la partición de test de cada idioma. Las medias y la velocidad se calculan a partir de las horas y palabras de la tabla 8.*

Idioma	Duración media	Duración mediana	Palabras por intervención	Palabras por minuto
Castellano	93 s	84 s	242	156
Inglés	89 s	74 s	223	151
Francés	104 s	97 s	284	164
Alemán	99 s	89 s	231	140

Una intervención típica dura entre un minuto y un minuto y tres cuartos, y las más largas superan los cuatro minutos. Para un sistema en diferido la duración es indiferente; para uno que reprocesa el audio acumulado, como se verá, es la variable decisiva, y el francés, con las intervenciones más largas y el habla más rápida, es el idioma en el que el límite de duración que impone el capítulo 4 deja fuera más audios. El segundo rasgo es el registro. Las intervenciones son discursos generalmente preparados pero pronunciados en directo, con las vacilaciones y reformulaciones propias del habla oral, la acústica de la sala y, a veces, la voz de la Presidencia al final del turno. No es habla espontánea ni habla leída en estudio, sino algo intermedio y más difícil que lo segundo.

3.5.3. La línea base del artículo original

El artículo de Europarl-ST evaluó sistemas de reconocimiento híbridos, que combinan redes neuronales profundas con modelos ocultos de Markov, entrenados de forma específica para cada idioma; en todos salvo el francés, el modelo acústico incluía capas recurrentes bidireccionales. En los cuatro pares usados aquí, sus tasas de error sobre el test oscilan entre el 15,0 % del castellano y el 24,3 % del francés, el idioma con menos recursos acústicos de los cuatro y el único sin esas capas. Son el punto de partida contra el que se miden los resultados del capítulo 4. Dos advertencias acompañan a esa comparación: los sistemas de 2020 no son modelos fundacionales, sino reconocedores especializados, y el conjunto de intervenciones evaluado puede no coincidir exactamente con el de este trabajo, porque cada par de idiomas tiene su propio test.

3.6. Crítica del estado del arte y propuesta

Los dos modelos llegan con evaluaciones serias. Voxtral Realtime muestra la curva de calidad frente al retardo en FLEURS, Common Voice y varios conjuntos en inglés (Mistral AI 2026); Qwen3-ASR compara su modo diferido con su modo en streaming en LibriSpeech y FLEURS (Shi et al. 2026); y la línea de trabajo que adapta modelos diferidos al tiempo real, con Whisper como caso más conocido, ha descrito los compromisos de trocear la entrada (Macháček, Dabre y Bojar 2023). Leídas juntas, sin embargo, estas evaluaciones dejan tres huecos.

El primero es de método. Cada artículo mide su modelo con su propia definición de latencia, y ninguno de los revisados compara un modelo nativamente causal con uno que reprocesa la entrada en el mismo punto de latencia algorítmica. El segundo es de coste. Las cifras publicadas son de calidad; el tiempo de cómputo por paso, su evolución a lo largo de una intervención larga y la memoria que exigen en una GPU doméstica no aparecen, o aparecen medidos en servidores con concurrencia. El tercero es de dominio. FLEURS y LibriSpeech son habla leída; un debate parlamentario multilingüe, con intervenciones de varios minutos, no lo es.

Este trabajo propone cubrir esos tres huecos a escala modesta: evaluar los dos modelos en el test de Europarl-ST en cuatro idiomas, primero en diferido y después en streaming; compararlos a igual latencia algorítmica midiendo también la latencia computacional y su crecimiento en una RTX 3060; y comprobar el resultado en un demostrador que transcribe la voz del usuario en directo. No es una aportación de arquitectura. Es una medida cuidadosa de lo que dos arquitecturas distintas cuestan y ofrecen cuando el tiempo real deja de ser una etiqueta y pasa a ser una restricción.

4. Evaluación experimental

Este capítulo recoge tres experimentos encadenados, y el orden importa porque cada uno nació de una pregunta que el anterior había dejado abierta. El primero (E1) mide la calidad de los dos modelos en diferido sobre el test completo de Europarl-ST en los cuatro idiomas. El segundo (E2) introduce el tiempo real: barre el parámetro de latencia de cada modelo sobre veinte intervenciones en castellano y mide, además del error, cuánto tarda cada paso y cómo evoluciona ese tiempo a lo largo de la intervención. El tercero (E3) vuelve a los cuatro idiomas para comparar, sobre los mismos audios, el modo diferido y el streaming de ambos modelos en el único punto de latencia en el que pueden compararse con justicia. La sección final discute qué mide cada métrica y qué no permiten afirmar los datos.

4.1. Entorno experimental

Todos los experimentos se ejecutaron en un ordenador de sobremesa con un procesador Intel Core i5-11400F, 16 GB de memoria y una tarjeta NVIDIA GeForce RTX 3060 de 12 GB. El código se escribió en Python sobre PyTorch (Paszke et al. 2019) y la biblioteca transformers (Wolf et al. 2020), en cuadernos de Jupyter organizados por experimento y por idioma.

Dos restricciones de software obligaron a desdoblar el entorno. La primera es un conflicto de versiones: el paquete oficial de Qwen3-ASR, qwen-asr, fija transformers en la versión 4.57.6, mientras que la clase de Voxtral Realtime solo existe en la rama 5 de esa biblioteca. Instalar ambos en el mismo entorno rompe uno de los dos modelos, de modo que cada uno tiene el suyo. La segunda es de sistema operativo: el modo streaming de Qwen3-ASR exige vLLM, que no tiene soporte oficial en Windows. Por eso E1 se ejecutó en Windows, y E2 y E3, junto con el demostrador, en una distribución Ubuntu 24.04 sobre el Subsistema de Windows para Linux (WSL2).8 Para que la comparación de latencias fuera homogénea, los dos modelos se midieron en WSL2 aunque Voxtral funcione también en Windows. La tabla 10 recoge las versiones.

Tabla 10. *Versiones del software empleado en cada entorno.*

Componente	**E1 (Windows 11)**	**E2, E3 y demostrador (WSL2, Ubuntu 24.04)**
Python	3.11.0	3.12.3
Entorno de Voxtral	torch 2.5.1, transformers 5.9.0, mistral-common 1.11.2	torch 2.5.1, transformers 5.17.0, mistral-common 1.11.7
Entorno de Qwen3-ASR	torch 2.5.1, transformers 4.57.6, qwen-asr 0.0.6	torch 2.9.1, transformers 4.57.6, qwen-asr 0.0.6, vLLM 0.14.0
Comunes	librosa 0.11.0, jiwer 4.0.0, openai-whisper 20250625	librosa 0.11.0, jiwer 4.0.0, openai-whisper 20250625
Controlador de la GPU	591.59 (el de Windows, compartido con WSL2)	591.59 (el de Windows, compartido con WSL2)

Hubo que fijar además la versión de librosa.9 Los audios de Europarl-ST están en formato AAC, que la biblioteca de sonido subyacente no sabe leer; la versión 0.11 recurre entonces a ffmpeg, y la 1.0 eliminó ese recurso, de modo que con ella la carga de

8 Documentación de WSL. Disponible en: https://learn.microsoft.com/es-es/windows/wsl/ (consulta: 19-10-2026).
9 librosa. Disponible en: https://librosa.org/ (consulta: 19-10-2026).

cualquier audio del corpus falla. Detalles así no aparecen en ningún resultado, pero consumieron una parte apreciable del tiempo del proyecto.

4.2. Metodología común

Los tres experimentos comparten el mismo tratamiento de los datos. Cada intervención se carga con librosa remuestreada a 16 kHz, la frecuencia que esperan los dos modelos. La referencia es el fichero transcription.tok de la intervención. Antes de comparar, hipótesis y referencia pasan por el normalizador básico del paquete de Whisper, que convierte a minúsculas, elimina la puntuación y el texto entre corchetes o paréntesis y unifica los espacios; el procedimiento se describe en la sección 2.5.1. El WER se calcula con jiwer sobre el conjunto completo, sumando errores y palabras de todas las intervenciones, y no como media de los WER individuales.

Hay una asimetría que no se pudo evitar. Qwen3-ASR acepta que se le indique el idioma y en todos los experimentos se le indicó; Voxtral Realtime no lo admite y lo detecta por sí mismo. Los resultados de Qwen3-ASR parten, por tanto, de una ayuda que Voxtral no tiene, y conviene recordarlo al leer los fallos de idioma de este último.

Cada cuaderno sigue la misma estructura en bloques: carga del modelo, lectura de la lista de test, transcripción de cada intervención con registro de los tiempos, guardado de las salidas en bruto en un CSV, normalización y guardado de un segundo CSV, y cálculo de las métricas. Separar las salidas en bruto de las normalizadas permitió repetir el análisis sin volver a ejecutar los modelos, y fue lo que hizo posible el análisis de errores de la sección 4.3.

4.3. E1: transcripción en diferido del test completo

4.3.1. Resultados por idioma

En E1 cada modelo recibe la intervención entera de una vez. Voxtral se ejecutó con generate sobre el audio completo y su retardo por defecto, 480 ms; Qwen3-ASR, con el motor de transformers en bfloat16 y un máximo de 4096 tokens nuevos, porque el valor por defecto de la biblioteca, 256, habría truncado las intervenciones largas. Se transcribieron las 729 intervenciones de los cuatro tests, 19,56 horas de grabación. La tabla 11 y la figura 6 recogen los resultados.

Tabla 11. *WER en diferido sobre el test completo de Europarl-ST.*

Idioma	Intervenciones	Palabras de referencia	Voxtral	Qwen3-ASR	Línea base (Iranzo-Sánchez et al. 2020)
Castellano	210	50.720	10,78 %	10,95 %	15,0 %
Inglés	127	28.320	15,35 %	11,50 %	17,2 %
Francés	166	47.109	9,68 %	15,06 %	24,3 %
Alemán	226	52.127	12,21 %	13,71 %	19,8 %
Media sin ponderar	729	178.276	12,01 %	12,81 %	

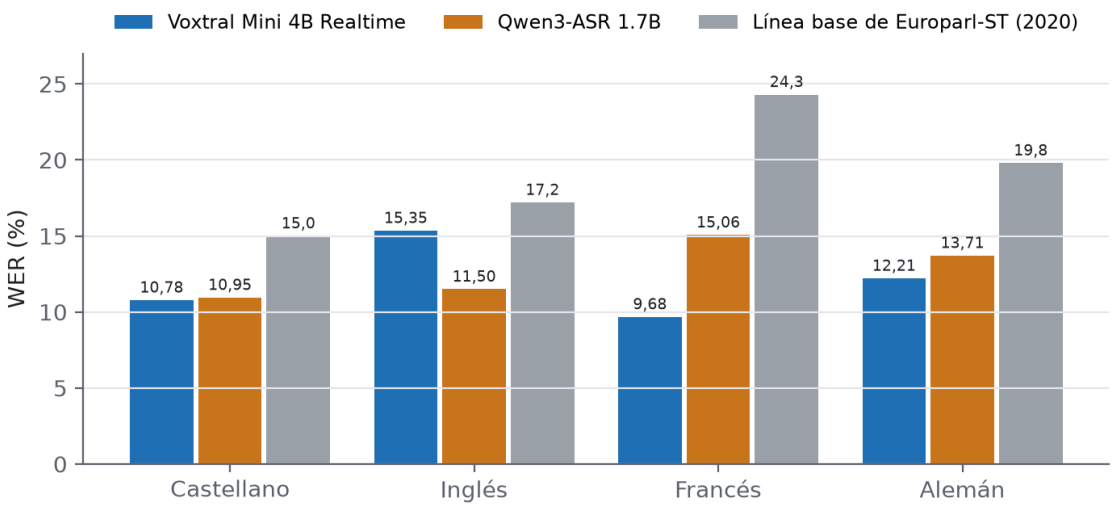


Figura 6. *WER en diferido de los dos modelos por idioma y línea base del artículo de Europarl-ST. Elaboración propia; línea base tomada de Iranzo-Sánchez et al. (2020).*

Los dos modelos fundacionales mejoran la línea base de 2020 en los cuatro idiomas, entre 1,85 y 14,62 puntos. Con la advertencia ya hecha en el capítulo 3 (la línea base corresponde a la primera versión del corpus y los listados de castellano e inglés son de la segunda), la mejora es clara: sistemas genéricos, sin ningún ajuste al dominio, transcriben el Parlamento Europeo mejor que los reconocedores entrenados específicamente para cada idioma hace seis años.

Entre ellos, en cambio, el resultado medio apenas los separa: 12,01 % frente a 12,81 %. El detalle por idioma cuenta otra historia. En castellano quedan empatados y en alemán Voxtral aventaja a Qwen3-ASR en 1,5 puntos, pero en inglés Qwen3-ASR es casi cuatro puntos mejor y en francés más de cinco puntos peor. Diferencias tan grandes y de signo contrario en idiomas con recursos abundantes resultan sospechosas, y el análisis de las transcripciones individuales confirmó la sospecha.

4.3.2. Fallos catastróficos

Una parte considerable de las diferencias no procede de la calidad general del reconocimiento, sino de unos pocos audios en los que uno de los modelos falla por completo. Los dos tienen fallos de este tipo, y no se parecen.

Voxtral colapsa en cinco intervenciones repartidas entre castellano, inglés y francés: devuelve una transcripción vacía, responde en un idioma equivocado u omite buena parte del discurso. El caso extremo es una intervención inglesa de 442 palabras de la que el modelo solo produce «Grazie, Presidente». Es un error de identificación de idioma que Qwen3-ASR no puede cometer en este montaje, porque se le dice en qué idioma escuchar.

Qwen3-ASR no colapsa en ninguno de los 729 audios, pero en uno entra en un bucle de repetición. Es una intervención francesa de 103 segundos (en.20110913.45.2-707-000) sin ninguna particularidad acústica: la transcripción empieza bien y, a mitad de la primera frase, el modelo se queda atrapado en la secuencia «et d'en constater la crise» y la repite hasta agotar el límite de tokens. Genera más de 2200 palabras frente a las

314 de la referencia; ese único audio aporta 2404 inserciones y el 37 % de todos los errores del francés.

¿Cuánto pesan estos casos en el orden de los modelos? Para saberlo, la tabla 12 repite el cálculo sin los audios con salida manifiestamente anómala. Es una cifra de diagnóstico, no un sustituto del resultado de la tabla 11, porque un sistema real no puede elegir en qué audios falla.

Tabla 12. *WER en diferido sin los audios con transcripción anómala (diagnóstico).*

Idioma	Voxtral	Qwen3-ASR
Castellano	9,98 %	10,95 %
Inglés	13,41 %	11,50 %
Francés	9,17 %	9,58 %
Alemán	12,21 %	13,71 %
Media	11,20 %	11,44 %

Depurados los casos patológicos, la diferencia media queda por debajo de tres décimas y el francés, que parecía el idioma más favorable a Voxtral, se reduce a cuatro décimas. Lo que distingue a los dos modelos en diferido no es la precisión, sino la frecuencia y la naturaleza de sus fallos: Voxtral se equivoca de idioma o se calla; Qwen3-ASR, en rarísimas ocasiones, no sabe parar.

4.3.3. Distribución y tipos de error

El WER del conjunto resume en una cifra lo que ocurre en cientos de intervenciones, y por construcción es sensible a las más largas y a las peores. La figura 7 muestra la distribución del WER de cada intervención por idioma y modelo, que permite ver lo que la media oculta.

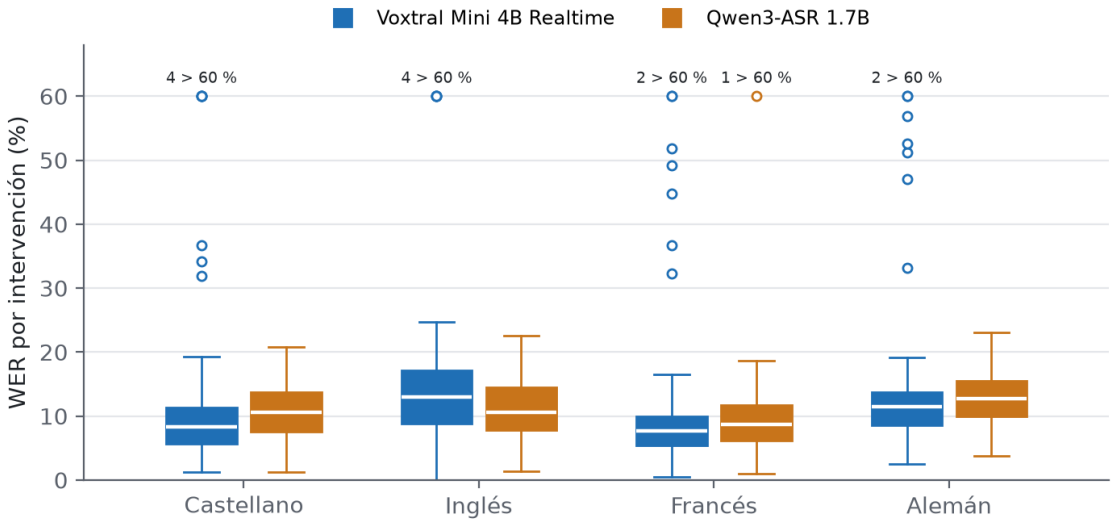


Figura 7. *Distribución del WER por intervención en diferido. Las cajas abarcan del primer al tercer cuartil, la línea blanca es la mediana y los círculos son las intervenciones por encima del 30 %; los valores superiores al 60 % se dibujan en el borde y se indica su número. Elaboración propia a partir de las salidas de E1.*

Las cajas de los dos modelos se solapan en los cuatro idiomas, pero sus medianas cuentan algo distinto de las tasas globales. La mediana del WER por intervención de Voxtral es menor que la de Qwen3-ASR en castellano (8,29 frente a 10,64 %), francés

(7,64 frente a 8,65 %) y alemán (11,42 frente a 12,72 %), y solo es mayor en inglés (12,99 frente a 10,60 %). En una intervención típica, Voxtral transcribe algo mejor. Lo que eleva su tasa global es la cola: dieciséis intervenciones por encima del 50 % de error, entre ellas las de salida vacía, frente a una sola de Qwen3-ASR, la del bucle de repetición, que con un 924,8 % no cabe en la escala. Qwen3-ASR es, por así decirlo, más regular: su peor intervención, si se excluye esa, no llega al 30 %.

La descomposición del error en sustituciones, eliminaciones e inserciones, en la tabla 13, señala de dónde procede esa diferencia.

Tabla 13. *Sustituciones (S), eliminaciones (D) e inserciones (I) en E1, en porcentaje sobre las palabras de referencia. La suma de cada terna es el WER de la tabla 11.*

Idioma	Voxtral S	Voxtral D	Voxtral I	Qwen3-ASR S	Qwen3-ASR D	Qwen3-ASR I
Castellano	3,08	4,77	2,93	4,98	3,01	2,97
Inglés	4,79	6,12	4,44	4,18	3,29	4,03
Francés	3,22	4,07	2,40	5,11	2,50	7,45
Alemán	4,11	3,70	4,40	6,23	3,17	4,30

Voxtral elimina más palabras que Qwen3-ASR en los cuatro idiomas, y la diferencia es mayor allí donde se concentran sus colapsos, el castellano y el inglés: las salidas vacías y los discursos omitidos se contabilizan como eliminaciones. Qwen3-ASR, en cambio, sustituye más palabras en tres de los cuatro idiomas, señal de que transcribe todo pero se equivoca más a menudo en la palabra concreta, y sus inserciones se disparan en francés por el bucle de repetición. Los dos perfiles son coherentes con lo descrito en la sección anterior: Voxtral falla callándose, Qwen3-ASR falla escribiendo.

4.3.4. El caso del inglés

El inglés es el idioma con más recursos de entrenamiento en casi cualquier modelo, y sin embargo es el peor de Voxtral. La comparación palabra a palabra de sus transcripciones con las referencias explica la anomalía con dos factores concretos.

El primero son tres audios fallidos: el de la respuesta en italiano, uno con salida vacía y otro en el que el modelo omite los dos primeros tercios del discurso. Concentran el 18 % de todos los errores del conjunto y elevan el WER en 2,4 puntos, efecto que se amplifica porque el test de inglés es el más pequeño de los cuatro. El segundo son discrepancias de formato que no son errores de reconocimiento: Voxtral escribe las formas contraídas (*don't*, *it's*, *we're*) donde las referencias usan siempre la forma desarrollada, y emplea la ortografía estadounidense (*program*, *defense*, *realize*) frente a la británica del corpus. Juntas suman 0,8 puntos. Descontados los tres fallos y unificadas contracciones y ortografía, el WER del inglés baja al 12,13 %, prácticamente el del alemán. La tabla 14 resume la descomposición.

Tabla 14. *Descomposición del WER de Voxtral en inglés en E1. Los efectos se estimaron por separado y su suma no coincide exactamente con la diferencia total, porque interactúan.*

Paso	Efecto en el WER	WER resultante
WER medido		15,35 %
Sin los tres audios fallidos	−2,4 puntos	
Con contracciones y ortografía unificadas	−0,8 puntos	
WER corregido (diagnóstico)		12,13 %

Los errores restantes son comunes a los cuatro idiomas y a los dos modelos: nombres propios y terminología parlamentaria, una tendencia a reformular el habla espontánea en lugar de transcribirla literalmente y, sobre todo, inserciones sistemáticas al final de muchas intervenciones. Proceden de la respuesta de la Presidencia, que el recorte por diarización dejó en el audio y que la referencia no contiene (sección 3.5.1). Un sistema que transcribe bien lo que oye es penalizado por ello.

4.3.5. Coste en diferido

La diferencia más visible entre los modelos en E1 no estuvo en el error, sino en el tiempo. La tabla 15 lo recoge junto con el factor de tiempo real (RTF), calculado aquí como tiempo total de proceso de cada idioma entre la duración de su audio.

Tabla 15. *Tiempo de proceso en diferido en Windows y RTF por idioma.*

Idioma	Audio	Voxtral	Qwen3-ASR	RTF Voxtral	RTF Qwen3-ASR	Cociente
Castellano	5,42 h	436 min	72 min	1,34	0,22	6,05
Inglés	3,13 h	307 min	36 min	1,63	0,19	8,53
Francés	4,80 h	479 min	80 min	1,66	0,28	5,99
Alemán	6,20 h	512 min	96 min	1,38	0,26	5,33
Total	19,56 h	1734 min	284 min			6,11

Qwen3-ASR transcribió el corpus completo en menos de cinco horas, entre 3,6 y 5,3 veces más rápido que el propio audio, con un pico de memoria de unos 4,7 GB. Voxtral necesitó casi 29 horas y en ningún idioma alcanzó el tiempo real; sus pesos ocupan además unos 8,25 GiB. Leída de forma aislada, la tabla invita a descartar Voxtral para cualquier uso en directo.

Esa lectura es precipitada. Un modelo causal que emite un token por cada trama de 80 ms hace, en diferido, exactamente el mismo número de pasos que en streaming: recibir el audio de golpe no le ahorra trabajo, porque su decodificador sigue avanzando trama a trama. Su RTF en diferido mide, en el fondo, cuánto cuesta cada paso en ese equipo y con ese sistema operativo, y en Windows ese coste resultó mayor que en Linux, como confirmarían E2 y las pruebas del demostrador. Qwen3-ASR, en cambio, procesa en diferido toda la intervención en una sola pasada del codificador. Comparar los dos con el RTF de E1 es comparar un modo de funcionamiento natural de uno con un modo impropio del otro. E2 plantea la comparación en los términos que importan.

4.4. E2: barrido de latencia en castellano

4.4.1. Diseño

E2 mide qué ocurre con la calidad, la latencia y el coste al mover el parámetro de latencia de cada modelo. En Voxtral se probaron retardos de 160, 320, 480, 960 y 1920 ms; en Qwen3-ASR, bloques de 0,48, 1, 2 y 4 segundos. El último retardo de Voxtral, 1920 ms, queda fuera de los valores admitidos por la ficha del modelo (múltiplos de 80 ms hasta 1200 ms, más 2400 ms), algo que se advirtió después de ejecutar el barrido; se presenta solo a título informativo.

El conjunto de evaluación son las veinte primeras intervenciones del test de castellano, con duraciones de 6 a 133 segundos, 29,8 minutos de grabación y 4789 palabras de referencia. No es una muestra aleatoria: se tomaron las primeras de la lista, un defecto que E3 corrige. Voxtral se ejecutó en streaming con transformers, entregando el audio

en pasos de 80 ms tal como haría un micrófono; Qwen3-ASR, con vLLM y su función streaming_transcribe, entregando bloques del tamaño configurado. Para que el motor de Qwen3-ASR arrancara en la RTX 3060 fue necesario fijar la fracción de memoria de la GPU que puede reservar en 0,88 y la longitud máxima de secuencia en 24.576 tokens.

De cada llamada al modelo se midió el tiempo de cómputo. A partir de esos tiempos se calculan las métricas definidas en la sección 2.5: la latencia computacional media por paso; la latencia total, suma de la algorítmica y la computacional; el crecimiento, cociente entre la latencia media del último tercio de las intervenciones y la del primero; el uso del paso, porcentaje del audio nuevo de cada paso que se consume en cómputo (80 ms en Voxtral y el tamaño del bloque en Qwen3-ASR); y el RTF. El WER se acompaña siempre de una versión sin anómalos, que excluye las transcripciones de menos de la mitad o más de 1,6 veces la longitud de la referencia, y del número de fallos, intervenciones que agotaron la longitud máxima de contexto sin producir transcripción.

4.4.2. Voxtral: coste constante y una banda fiable

La tabla 16 resume el barrido de Voxtral.

Tabla 16. *Barrido de latencia de Voxtral sobre veinte intervenciones en castellano (WSL2).*

Retardo	Lat. comp.	Lat. total	Crecim.	RTF	Uso del paso	WER	WER sin anómalos	Anómalos
160 ms	71,5 ms	232 ms	1,01	0,89	89 %	16,98 %	15,81 %	1
320 ms	70,2 ms	390 ms	1,03	0,87	88 %	13,97 %	8,32 %	3
480 ms	67,2 ms	547 ms	1,00	0,83	84 %	9,69 %	9,69 %	0
960 ms	68,0 ms	1028 ms	1,01	0,83	85 %	7,41 %	7,41 %	0
1920 ms	69,6 ms	1990 ms	1,03	0,84	87 %	19,98 %	9,21 %	3

La latencia computacional se mantiene en torno a 68 ms sea cual sea el retardo, y el crecimiento no se separa de 1 en ninguna configuración. Es la firma de un estado incremental real: las cachés de la convolución y de la atención hacen que procesar el paso número mil cueste lo mismo que procesar el primero. El modelo consume entre el 84 y el 89 % de cada paso de 80 ms. El margen frente al tiempo real es estrecho, entre el 11 y el 16 %, pero no se erosiona con la duración, y eso permite sostener el tiempo real de forma indefinida.

El WER exige más cuidado. La tasa global sube con fuerza en 320 y 1920 ms, pero sin las transcripciones anómalas se queda entre el 7,41 y el 9,69 % en todo el rango de 320 a 1920 ms; la mejor cifra, 7,41 %, corresponde a 960 ms. Solo 160 ms degrada de verdad el reconocimiento: el 15,81 % se mantiene incluso descontando su único caso anómalo, y es coherente con lo que cabe esperar de un modelo que decide cada palabra con apenas dos tramas de contexto posterior.

Las tasas altas de 320 y 1920 ms se deben a truncamientos. En ellos el modelo sigue bien el principio del discurso, corta la generación antes de que termine la intervención y remata con una fórmula de cierre, como si emitiera el final de la transcripción antes de tiempo. En 1920 ms la explicación es inmediata, porque ese retardo no forma parte de los admitidos. En 320 ms, que sí lo es, los truncamientos no tienen explicación por ahora; se concentran en cuatro intervenciones concretas y no siguen un patrón claro,

porque 320 ms trunca más que 160 ms y un mismo audio puede transcribirse bien a 480 y 960 ms y truncarse a 160 y 320 ms. Entre 480 y 960 ms no hubo ninguna transcripción anómala en los veinte audios: es la banda en la que Voxtral resultó fiable.

4.4.3. Qwen3-ASR y el techo de contexto

La tabla 17 resume el barrido de Qwen3-ASR.

Tabla 17. *Barrido de latencia de Qwen3-ASR sobre las mismas veinte intervenciones (WSL2). Los fallos quedan fuera del WER y de las latencias.*

Bloque	Lat. comp.	Lat. total	Crecim.	RTF	Uso del bloque	WER	WER sin anómalos	Anómalos	Fallos
0,48 s	351,5 ms	832 ms	3,21	0,74	73 %	154,72 %	11,73 %	2	3
1 s	321,5 ms	1321 ms	2,17	0,33	32 %	8,56 %	8,56 %	0	1
2 s	354,6 ms	2355 ms	1,77	0,19	18 %	8,90 %	8,90 %	0	0
4 s	432,6 ms	4433 ms	1,36	0,12	11 %	7,93 %	7,93 %	0	0

El comportamiento es el inverso del de Voxtral. El crecimiento llega a 3,21 con bloques de 0,48 s, de modo que el último tercio de las intervenciones se procesa más de tres veces más despacio que el primero, y baja de forma ordenada al ampliar el bloque, hasta 1,36 con 4 s. Es exactamente lo que predice el mecanismo descrito en la sección 3.3.2: si cada paso reprocesa todo el audio acumulado, el coste de un paso crece con la posición dentro de la intervención, y tanto más deprisa cuanto más pasos hay, es decir, cuanto menor es el bloque.

El 154,72 % de la primera fila no es un error de cálculo. Una intervención de 122 segundos generó 5473 palabras frente a las 331 de su referencia, una salida dieciséis veces más larga que el original y un WER individual del 1584 %. Sin ese audio la tasa baja al 17,67 %, y sin un segundo caso anómalo, al 11,73 %.

Los fallos tienen una causa precisa. vLLM reserva por adelantado la fracción de memoria indicada, descuenta pesos, activaciones y grafos de ejecución, y dedica el resto a la caché de claves y valores (Kwon et al. 2023); además, se niega a arrancar si esa caché no basta para atender una petición de la longitud máxima declarada. En la RTX 3060, con el 88 % de los 12 GB, los pesos ocupan 3,87 GB y quedan 3,05 GB de caché, equivalentes a 28.560 tokens. Por eso el límite se fijó en 24.576 tokens; con el valor por defecto del modelo, 65.536, el motor no arranca. Como la secuencia que recibe Qwen3-ASR incluye todo el audio acumulado y el texto ya generado, una intervención suficientemente larga acaba superando ese límite. Con bloques de 0,48 s fallaron tres intervenciones, de 124, 127 y 129 segundos, y otras dos, de 122 y 133 segundos, terminaron con una salida degenerada; con bloques de 1 s falló una, de 130 segundos; con 2 y 4 s no hubo incidencias. Ninguna intervención por debajo de 110 segundos dio problemas.

La consecuencia es incómoda para cualquier uso en directo. En Qwen3-ASR la latencia que se pide y la duración que admite una intervención tiran en sentidos opuestos: bajar la latencia obliga a dar más pasos, cada paso reprocesa lo acumulado y alarga el contexto, y el techo llega antes. En un demostrador, donde nadie sabe cuánto va a

hablar el usuario, eso no es un parámetro que se ajuste, sino un límite que hay que imponer.

4.4.4. Comparación a igual latencia

La figura 8 sitúa las dos curvas en el mismo plano: WER sin anómalos frente a latencia total. Los dos modelos solo coinciden en latencia algorítmica en un punto, 480 ms, valor nativo de Voxtral y bloque más pequeño ensayado con Qwen3-ASR. La tabla 18 compara ese punto.

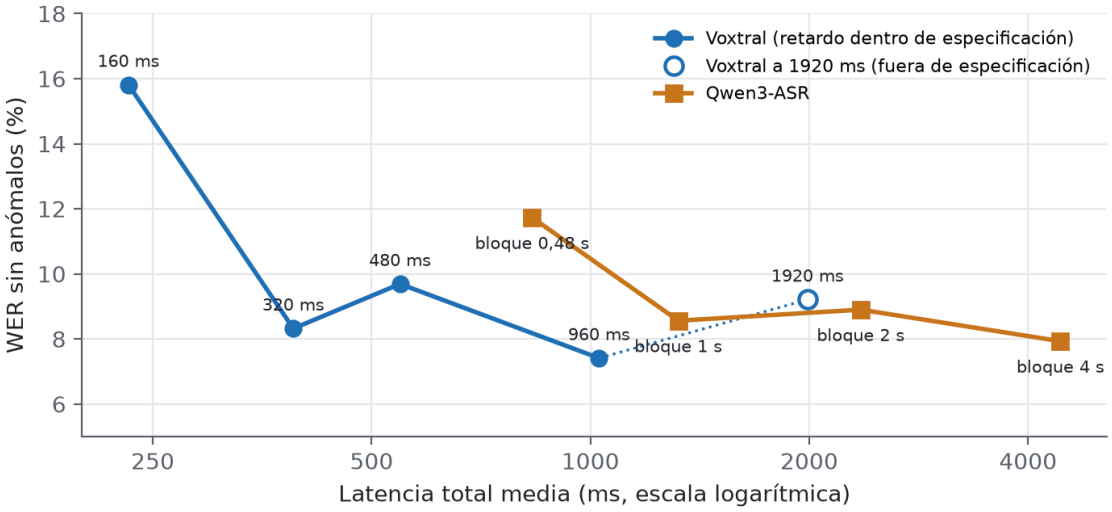


Figura 8. *WER sin anómalos frente a latencia total media de cada configuración en castellano. La línea de puntos une el último valor admitido de Voxtral con el valor fuera de especificación. Elaboración propia.*

Tabla 18. *Comparación de los dos modelos a 480 ms de latencia algorítmica.*

	Voxtral	**Qwen3-ASR**
Latencia algorítmica	480 ms	480 ms
Latencia computacional	67 ms	352 ms
Latencia total	547 ms	832 ms
Uso del paso	84 %	73 %
Crecimiento	1,00	3,21
WER	9,69 %	154,72 %
WER sin anómalos	9,69 %	11,73 %
Intervenciones con incidencia	0 de 20	5 de 20

A igual latencia algorítmica, Voxtral gasta en cada paso unas cinco veces menos tiempo de cómputo, ofrece una latencia total notablemente menor y no produce ninguna transcripción defectuosa, mientras que Qwen3-ASR falla o degenera en la cuarta parte del conjunto. Incluso sobre los audios en los que Qwen3-ASR funciona, Voxtral conserva una ventaja de dos puntos. El empate de E1 se rompe en cuanto el tiempo real entra en escena.

El uso del paso, leído solo, parece favorecer a Qwen3-ASR: 73 % frente a 84 %. Es una media sobre toda la intervención, y la media oculta la tendencia. La figura 9 muestra el uso del paso en el último tercio de cada intervención frente a su duración. Los puntos de Voxtral forman una banda horizontal entre el 79 y el 90 % sea cual sea la duración. Los de Qwen3-ASR suben con ella: en torno al 20 % en la intervención de 6 segundos, por encima del 100 % a partir de unos 108 segundos y hasta el 135 % en la de 133. Por

encima del 100 % el sistema tarda en procesar cada bloque más de lo que dura el bloque, y el retraso se acumula sin remedio.

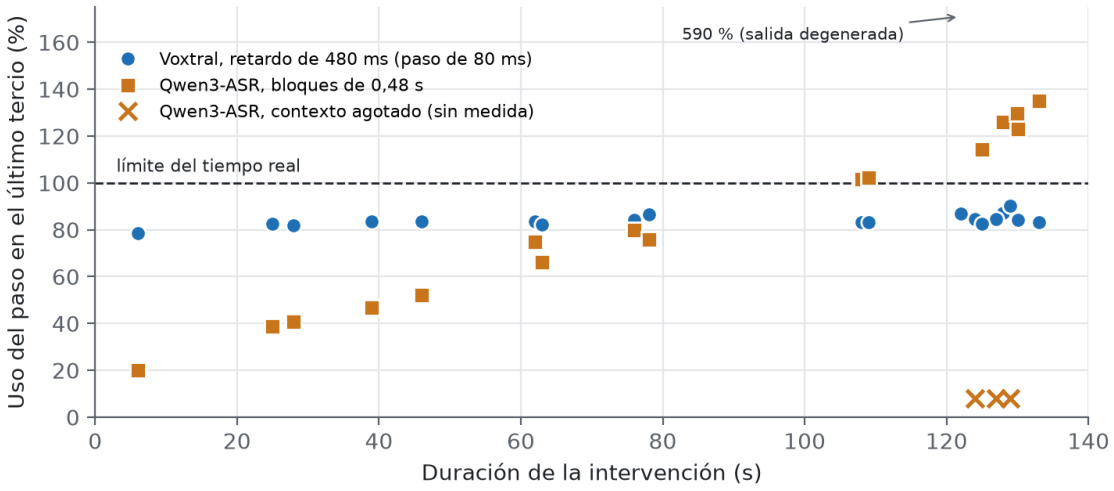


Figura 9. *Uso del paso en el último tercio de cada intervención frente a su duración, a 480 ms de latencia algorítmica. Las cruces marcan las intervenciones en las que Qwen3-ASR agotó el contexto. Elaboración propia a partir de los tiempos medidos en E2.*

Qwen3-ASR a 480 ms solo sostiene el tiempo real en intervenciones cortas. Con bloques de 1 a 4 segundos, en cambio, es un sistema excelente: WER entre el 7,93 y el 8,90 %, una sola incidencia (con bloques de 1 s) y un RTF muy inferior al de Voxtral. Su terreno natural es el de las latencias que se miden en segundos.

4.4.5. Variabilidad del tiempo de cómputo

Las latencias anteriores son medias, y un sistema en directo no se juzga solo por su media: un paso lento retrasa todos los que vienen detrás hasta que el sistema recupera el ritmo. La tabla 19 añade el percentil 90 del tiempo de cómputo por paso, calculado en cada intervención y promediado sobre las veinte.

Tabla 19. *Tiempo de cómputo por paso en E2: media, percentil 90 y medias del primer y del último tercio de cada intervención, promediados sobre las intervenciones completadas.*

Modelo y configuración	Media	Percentil 90	Primer tercio	Último tercio
Voxtral, 160 ms	71,5 ms	94,5 ms	70,5 ms	71,3 ms
Voxtral, 320 ms	70,2 ms	89,3 ms	69,3 ms	71,1 ms
Voxtral, 480 ms	67,2 ms	81,2 ms	67,2 ms	67,2 ms
Voxtral, 960 ms	68,0 ms	82,9 ms	67,3 ms	68,2 ms
Voxtral, 1920 ms	69,6 ms	88,1 ms	68,4 ms	70,4 ms
Qwen3-ASR, bloque de 0,48 s	351,5 ms	535,0 ms	168,7 ms	541,2 ms
Qwen3-ASR, bloque de 1 s	321,5 ms	473,5 ms	201,0 ms	437,0 ms
Qwen3-ASR, bloque de 2 s	354,6 ms	507,0 ms	251,5 ms	444,5 ms
Qwen3-ASR, bloque de 4 s	432,6 ms	599,1 ms	358,0 ms	487,5 ms

En Voxtral, el percentil 90 supera los 80 ms del paso en todas las configuraciones: uno de cada diez pasos tarda más de lo que dura el audio que incorpora. No por ello el sistema pierde el tiempo real, porque la media queda por debajo y los pasos rápidos compensan a los lentos; el audio que llega mientras tanto espera en la cola y se procesa

en cuanto el modelo queda libre. Lo que sí ocurre es que la latencia percibida oscila en unas decenas de milisegundos alrededor de su media. Es una cifra pequeña frente a los 480 ms de retardo, pero recuerda que un margen medio del 16 % no es un margen garantizado en cada paso.

En Qwen3-ASR, la tabla hace visible el crecimiento: con bloques de 0,48 s, el tiempo medio de un paso pasa de 169 ms en el primer tercio a 541 ms en el último, por encima de la duración del bloque. Con bloques de 4 s, el primer tercio ya parte de 358 ms, porque cada paso procesa más audio, pero el aumento relativo es mucho menor.

4.5. E3: diferido frente a streaming en los cuatro idiomas

4.5.1. Diseño y muestreo

E3 responde a dos preguntas que E1 y E2 dejaban abiertas: cuánto pierde cada modelo al pasar de diferido a streaming, y si lo observado en castellano se mantiene en los demás idiomas. El streaming se evaluó a 480 ms en los dos modelos (transcription_delay_ms=480 en Voxtral y bloques de 0,48 s en Qwen3-ASR), el único punto comparable.

De cada idioma se tomaron 20 intervenciones mediante un muestreo aleatorio reproducible, con semilla 42, limitado a audios de 90 segundos como máximo. El límite no es arbitrario: con bloques de 0,48 s, E2 mostró que Qwen3-ASR agota su contexto en las intervenciones de más de 120 segundos, y en un primer intento sin límite una sola intervención de 142 segundos llevaba más de dos horas sin terminar. Los dos modelos procesaron exactamente los mismos audios, en el mismo equipo y en WSL2. La tabla 20 describe la muestra.

Tabla 20. *Muestra de E3: veinte intervenciones por idioma de hasta 90 s.*

Idioma	Duraciones	Grabación	Palabras de referencia
Castellano	26 a 89 s	24,0 min	3740
Inglés	29 a 87 s	22,0 min	3471
Francés	61 a 89 s	24,4 min	3894
Alemán	16 a 81 s	20,3 min	2797
Total	16 a 89 s	90,7 min	13.902

4.5.2. Degradación por idioma

La tabla 21 y la figura 10 recogen los resultados.

Tabla 21. *WER en diferido y en streaming a 480 ms, y degradación en puntos. El streaming de Qwen3-ASR en inglés se calcula sobre 19 intervenciones.*

Idioma	Voxtral diferido	Voxtral streaming	Qwen3-ASR diferido	Qwen3-ASR streaming	Degradación Voxtral	Degradación Qwen3-ASR
Castellano	7,67 %	7,67 %	7,46 %	10,80 %	0,00	+3,34
Inglés	11,70 %	11,70 %	7,78 %	15,57 %	0,00	+7,79
Francés	7,68 %	7,65 %	6,21 %	11,04 %	−0,03	+4,83
Alemán	13,34 %	13,37 %	10,26 %	12,19 %	+0,03	+1,93
Media	10,10 %	10,10 %	7,93 %	12,40 %	0,00	+4,47

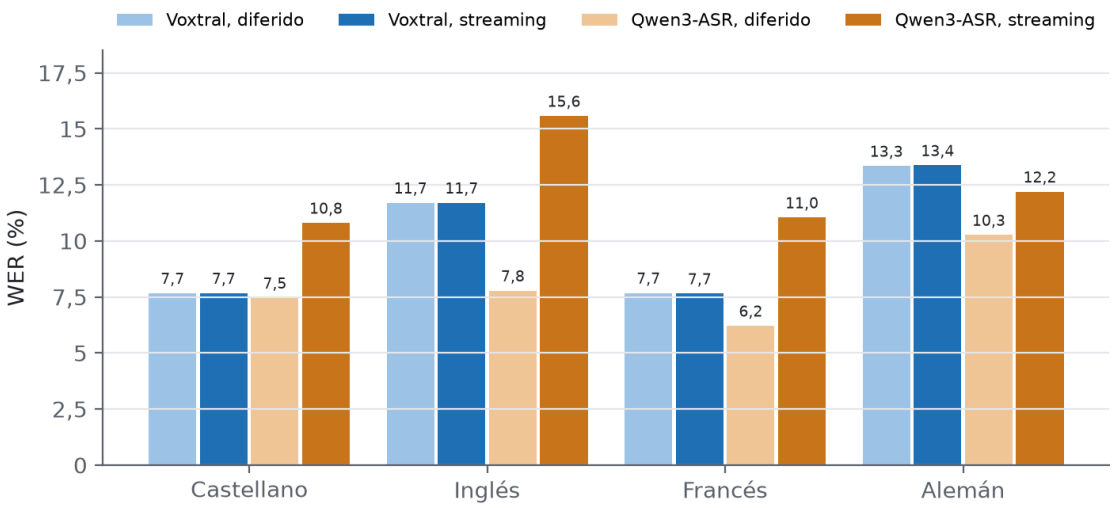


Figura 10. *WER en diferido y en streaming a 480 ms de los dos modelos en los cuatro idiomas. Elaboración propia.*

Voxtral no pierde nada al pasar a streaming. En 77 de las 80 intervenciones la transcripción en streaming es idéntica, palabra por palabra, a la diferida, y en las tres restantes la diferencia es de unas pocas palabras que mueven la tasa en centésimas. No es un error de medida: las dos funciones siguen caminos distintos, una entrega el audio completo y la otra lo trocea en pasos de 80 ms apoyándose en las cachés, y sus tiempos de ejecución también difieren. La explicación está en la arquitectura. Voxtral Realtime es causal por construcción y nunca mira más allá de su retardo, reciba el audio de golpe o a trozos; su modo diferido no dispone de más contexto que el streaming, sino que hace el mismo cálculo en una sola llamada. Que las salidas coincidan confirma además que su implementación del streaming es exacta.

El resultado obliga a reformular la pregunta. Para Voxtral la degradación es nula por definición, no porque su streaming sea especialmente bueno. Medir cuánto le cuesta en precisión la restricción del tiempo real exigiría compararlo con un modelo de la misma familia que vea la intervención completa, como Voxtral Mini, la versión de la familia que procesa la grabación completa con un codificador derivado del de Whisper y que admite hasta 40 minutos de audio (Liu et al. 2025); esa comparación queda fuera de este trabajo.

Qwen3-ASR sí se degrada, 4,47 puntos de media, desde 1,93 en alemán hasta 7,79 en inglés. Su modo diferido ve la intervención entera y puede usar el contexto posterior para decidir cada palabra; en streaming solo dispone de lo recibido hasta el momento, revisa sus últimos tokens en cada paso y reprocesa todo lo acumulado. La figura 11 compara, intervención por intervención, el WER de los dos modos.

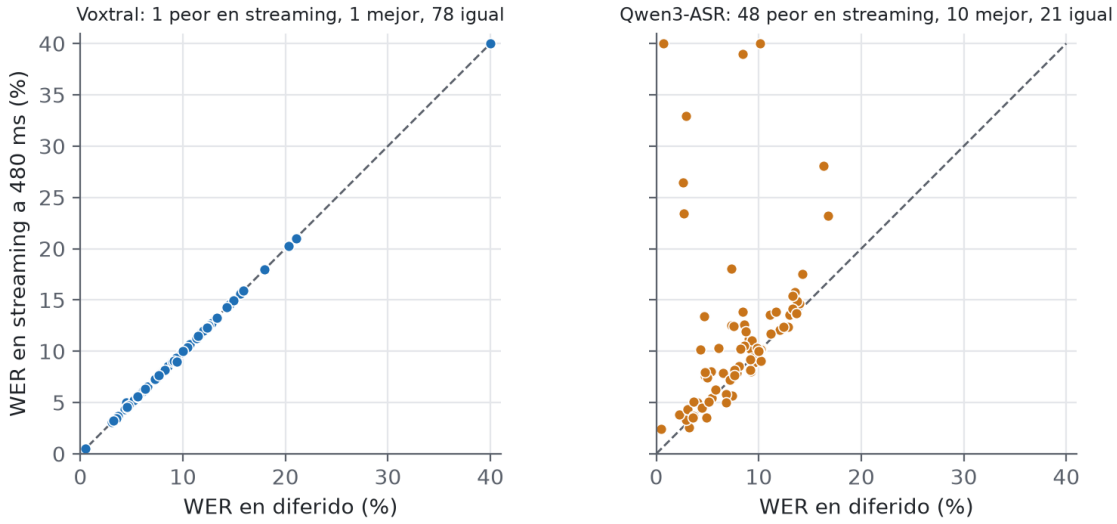


Figura 11. *WER en streaming frente a WER en diferido de cada intervención de E3. Los puntos sobre la diagonal empeoran al pasar a streaming. Los valores superiores al 40 % se dibujan en el borde. Elaboración propia.*

Los puntos de Voxtral se alinean sobre la diagonal: 78 de las 80 intervenciones tienen exactamente el mismo WER en los dos modos. Los de Qwen3-ASR se dispersan por encima de ella. De sus 79 intervenciones completadas, 48 empeoran en streaming, 21 quedan igual y 10 mejoran ligeramente. La degradación, por tanto, no se debe a unos pocos casos extremos, aunque los haya (cinco intervenciones superan el 20 % en streaming partiendo de menos del 10 % en diferido): es un desplazamiento general de la nube de puntos. El streaming de Qwen3-ASR pierde algo en la mayoría de las intervenciones y mucho en unas pocas.

Los modelos se cruzan. En diferido Qwen3-ASR es mejor en los cuatro idiomas, con una media del 7,93 % frente al 10,10 % de Voxtral; en streaming a 480 ms Voxtral gana en tres de los cuatro, con una media del 10,10 % frente al 12,40 %. La excepción es el alemán, donde Qwen3-ASR en streaming (12,19 %) sigue por delante de Voxtral (13,37 %): coinciden el idioma más difícil para Voxtral y aquel en el que Qwen3-ASR menos pierde al pasar al tiempo real.

El coste confirma lo visto en E2, como muestra la tabla 22. Voxtral tiene un RTF medio de entre 0,93 y 0,98 en ambos modos, coherente con que hace el mismo cálculo. Qwen3-ASR pasa de un RTF de entre 0,04 y 0,06 en diferido a uno de entre 0,45 y 0,53 en streaming, unas diez veces más, por el reprocesado. Las cifras de Voxtral en WSL2 quedan además por debajo de las de E1 en Windows, entre 1,34 y 1,66. La comparación es solo orientativa, porque en E1 el RTF se calculó con el tiempo total de cada idioma y aquí es una media por intervención, pero apunta en la misma dirección que la diferencia de coste por paso entre los dos sistemas operativos medida en el demostrador (sección 5.5).

Tabla 22. *RTF medio por intervención en E3 (WSL2).*

Idioma	Voxtral diferido	Voxtral streaming	Qwen3-ASR diferido	Qwen3-ASR streaming
Castellano	0,95	0,94	0,06	0,52
Inglés	0,96	0,98	0,04	0,45
Francés	0,93	0,94	0,05	0,53
Alemán	0,94	0,94	0,06	0,49

4.5.3. Incidencias

Qwen3-ASR tuvo dos incidencias, ambas en inglés y ambas en streaming. Una intervención de 87 segundos agotó la longitud máxima de contexto, pese a estar por debajo del límite de 90 segundos: el techo depende también de la densidad del habla, porque el texto generado forma parte de la secuencia, y no solo de la duración. Otra se completó con una salida anómala, de menos de la cuarta parte de la longitud de la referencia; sin ella, el WER de Qwen3-ASR en streaming en inglés baja del 15,57 al 12,06 %.

Voxtral tuvo una: en una intervención inglesa de 29 segundos sigue bien el principio, se aparta después de lo que se dice y termina con una fórmula de cierre, quedándose en el 43 % de las palabras de la referencia. Es el mismo truncamiento observado en E2 fuera de la banda de 480 a 960 ms, ahora dentro de ella; la salida es idéntica en diferido y en streaming, y Qwen3-ASR transcribe el mismo audio sin problemas. Sin ese caso, el WER de Voxtral en inglés baja del 11,70 al 10,39 %. Ni siquiera la configuración nativa está completamente libre de este fallo.

4.6. Discusión

4.6.1. Lo que mide cada métrica

Si este trabajo se hubiera detenido en E1, su conclusión habría sido que Qwen3-ASR es el candidato evidente para transcribir en tiempo real: la misma precisión que Voxtral y un RTF seis veces menor. E2 dice lo contrario. ¿Cómo pueden dos medidas del coste ordenar los modelos al revés?

Porque no miden lo mismo. El RTF responde a una pregunta de capacidad: cuánto cómputo hace falta para procesar una hora de audio. La latencia responde a una pregunta de experiencia: cuánto tarda en aparecer una palabra después de pronunciada. Qwen3-ASR gasta poco cómputo en total, pero tiene que esperar a acumular un bloque antes de emitir y, cuando el bloque es pequeño, repite trabajo en cada paso; Voxtral trabaja casi sin pausa, paso a paso, y por eso su RTF es alto aunque cada palabra aparezca pronto. En las cuatro configuraciones de Qwen3-ASR el RTF es menor que el de Voxtral, y en las cuatro la latencia total es mayor que la de Voxtral a 480 ms. Un sistema en directo vive en la segunda dimensión, no en la primera. El RTF sirve para dimensionar un servidor; para elegir un modelo de transcripción en vivo, por sí solo, habría llevado a la decisión equivocada.

Tampoco la latencia media basta. Un sistema que hoy consume el 73 % de su presupuesto de tiempo puede consumir el 135 % dentro de dos minutos si su coste crece con la intervención. El uso del paso dice si el sistema llega a tiempo ahora; el crecimiento dice si seguirá llegando. Solo juntos permiten afirmar que un sistema sostiene el tiempo real de forma indefinida, y esa es la combinación que se propone

como criterio práctico para evaluar sistemas de transcripción en directo. Medir el instante y no la trayectoria es fotografiar un río y concluir que no se mueve.

Queda la cuestión del error. El WER global es la cifra que se publica y es extraordinariamente sensible a los fallos raros: un solo audio convirtió el 11,73 % de Qwen3-ASR en un 154,72 %, y cinco colapsos de Voxtral explican casi toda su desventaja en inglés y su ventaja aparente en francés. Por eso aquí se ha presentado siempre junto a una versión depurada y al recuento de incidencias. Ninguna de las tres cifras es la verdadera por separado; la tasa global dice cuánto se equivoca el sistema tal como es, la depurada cuánto reconoce cuando funciona, y el recuento, cuántas veces deja de funcionar.

4.6.2. Limitaciones

Las conclusiones anteriores descansan sobre muestras pequeñas. E2 usa veinte intervenciones y 4789 palabras en un solo idioma, y E3, veinte intervenciones por idioma con entre 2797 y 3894 palabras; bastan para ver tendencias tan marcadas como las descritas, pero no para dar cifras por idioma con precisión de décimas, y un solo audio anómalo mueve la tasa de un idioma más de un punto. E2 se limita al castellano y no tomó una muestra aleatoria.

El límite de 90 segundos de E3 deja fuera aproximadamente la mitad de cada test: las duraciones medianas son de 84 segundos en castellano, 74 en inglés, 97 en francés y 89 en alemán. Ese sesgo favorece a Qwen3-ASR, porque excluye justamente las intervenciones en las que su streaming funciona peor. E3, además, evalúa un único punto de latencia; con bloques de uno a cuatro segundos la comparación sería distinta.

El montaje introduce dos asimetrías. Qwen3-ASR recibe el idioma y Voxtral no. Y bajo WSL2 vLLM desactiva la memoria anclada en las transferencias entre el procesador y la GPU, lo que penaliza la latencia computacional de Qwen3-ASR; las diferencias observadas son mucho mayores que ese efecto, pero existe.

Por último, las latencias computacionales y los RTF son propios de una RTX 3060 en WSL2. Con hardware más potente el margen de Voxtral frente al tiempo real crecería y el techo de contexto de Qwen3-ASR se alejaría. Lo que no cambiaría es la forma de las curvas: la constancia del coste de Voxtral y el crecimiento del de Qwen3-ASR proceden de sus arquitecturas, no del equipo.

4.6.3. Implicaciones para un sistema en directo

¿Qué debería hacer, a la vista de estos resultados, quien tenga que montar un sistema de subtitulado en directo con un modelo fundacional y un hardware modesto? Los datos permiten algunas recomendaciones con un alcance mayor que el de los dos modelos evaluados.

La primera es decidir antes el régimen de latencia. Si el uso admite varios segundos de espera, como la transcripción de una reunión que se consulta mientras avanza, un modelo diferido adaptado con bloques grandes ofrece una precisión excelente y un coste bajo, y sus limitaciones apenas se manifiestan. Si el uso exige seguir la voz con menos de un segundo de retraso, como un subtítulo, la arquitectura importa más que la precisión publicada, y un modelo diseñado para el streaming parte con ventaja.

La segunda es medir la trayectoria y no el promedio. Una prueba con intervenciones de treinta segundos habría declarado apto a Qwen3-ASR con bloques de 0,48 s; las intervenciones de dos minutos lo desmienten. Cualquier evaluación de un sistema en directo debería incluir entradas tan largas como las que vaya a recibir en servicio, y comprobar que el coste por paso al final es el mismo que al principio.

La tercera es poner límites explícitos. Un sistema cuya memoria o cuyo coste crecen con la duración necesita un tope de duración, un corte de sesión o un reinicio periódico del estado, y es preferible que ese tope sea una decisión de diseño visible para el usuario a que sea un fallo por agotamiento de memoria. El demostrador del capítulo 5 aplica esta lección.

La cuarta tiene que ver con el idioma. En un entorno multilingüe, fijar el idioma elimina toda una clase de fallos, los de identificación, que en este trabajo explican algunos de los colapsos de Voxtral y un error de Qwen3-ASR en el demostrador. Cuando el idioma del orador se conoce, como en un parlamento con turnos asignados, conviene indicarlo al modelo si lo admite.

Ninguna de estas recomendaciones es una regla universal. Todas proceden de veinte intervenciones por idioma y de una sola tarjeta gráfica, y su valor es el de una hipótesis bien fundada que un despliegue real debería confirmar.

5. Desarrollo del demostrador

Los experimentos del capítulo 4 transcriben ficheros: el audio existe completo en el disco aunque se entregue al modelo trozo a trozo. Un demostrador debe transcribir la voz de una persona mientras habla, en un navegador, sin que nada esté preparado de antemano. Este capítulo describe cómo se pasó de un cuaderno a una aplicación que cualquiera con el enlace puede probar: los requisitos, las alternativas consideradas, la arquitectura elegida, su implementación, las pruebas y sus limitaciones.

5.1. Requisitos

La tabla 23 recoge los requisitos, fijados a partir de la propuesta del trabajo y de las conclusiones de los experimentos.

Tabla 23. *Requisitos del demostrador.*

Código	Requisito	Tipo
RF1	Capturar audio del micrófono desde el navegador y mostrar la transcripción mientras se habla	Funcional
RF2	Transcribir los cuatro idiomas del corpus	Funcional
RF3	Permitir elegir la latencia antes de empezar	Funcional
RF4	Reproducir intervenciones de Europarl-ST a velocidad real, transcribirlas en directo y mostrar la referencia y el WER al terminar	Funcional
RF5	Permitir elegir entre los dos modelos evaluados	Funcional
RF6	Mostrar en directo métricas de coste: cómputo por paso, RTF y audio procesado	Funcional
RNF1	Sostener el tiempo real en la RTX 3060 disponible	No funcional
RNF2	Poder probarse de forma remota, sin instalar nada	No funcional
RNF3	No generar coste económico	No funcional
RNF4	Restringir el acceso y no almacenar el audio de los usuarios	No funcional

RF4 merece una justificación. Una demostración con micrófono convence de que el sistema funciona, pero no permite saber cuánto se equivoca, porque no hay referencia. Las intervenciones de Europarl-ST la tienen, y reproducirlas a su velocidad natural mientras se transcriben enlaza el demostrador con los experimentos: el mismo audio que produjo una cifra en el capítulo 4 se puede ver transcribirse en pantalla.

5.2. Soluciones posibles y solución elegida

Se consideraron cuatro maneras de llevar el modelo a un navegador ajeno. La tabla 24 las compara frente a los requisitos que más las discriminan.

Tabla 24. *Alternativas para el despliegue del demostrador. Precios y condiciones consultados el 19-10-2026.*

Alternativa	Coste	Tiempo real con Voxtral	Acceso remoto	Problema principal
OBS Studio con el complemento LocalVocal	Nulo	No aplicable	No	Solo ejecuta modelos Whisper en formato GGML
Space de Hugging Face con GPU dedicada	0,40 $/h (T4) o 0,80 $/h (L4) mientras está activo	Sí	Sí	Coste por hora y plan de pago para alojar el Space
Space de Hugging Face con ZeroGPU	Nulo	No	Sí	GPU asignada por llamada, cuota diaria de minutos
Gradio en el equipo propio con enlace temporal	Nulo	Sí	Sí	Depende de que el PC esté encendido

LocalVocal es un complemento de OBS Studio que subtitula en local con Whisper ejecutado mediante whisper.cpp y admite cualquier modelo en el formato GGML de esa biblioteca,10 pero no Voxtral ni Qwen3-ASR. Quedó descartado sin más análisis.

Los Spaces de Hugging Face alojan aplicaciones con GPU dedicada. Una NVIDIA T4 con 16 GB cuesta 0,40 dólares por hora y una L4 con 24 GB, 0,80; se factura por minuto mientras el Space está arrancado o en marcha, se use o no, aunque puede configurarse para que se duerma por inactividad, y la documentación advierte de que crear un Space que ejecute código requiere un plan de pago.11 Técnicamente es la opción más limpia. Económicamente contradice RNF3, y el coste no estaría acotado: cada sesión de prueba del tutor o del tribunal se pagaría.

ZeroGPU es la variante gratuita: las cuentas personales pueden alojar hasta dos Spaces que reciben una GPU compartida solo mientras se ejecuta una función marcada para ello, con una duración por defecto de 60 segundos por llamada y una cuota diaria de cinco minutos para un usuario con cuenta gratuita y de dos minutos para uno sin identificar.12 El modelo de uso es el de una petición que se atiende y se libera, justo lo contrario de una sesión de streaming, que mantiene el modelo y sus cachés vivos mientras el usuario habla. Con cinco minutos diarios, además, no cabría ni una intervención larga del Parlamento.

La solución elegida fue la cuarta: ejecutar la aplicación en el propio equipo, con su RTX 3060, y publicarla mediante el enlace temporal que genera Gradio (Abid et al. 2019). Gradio es una biblioteca de Python para construir interfaces web de modelos de aprendizaje automático;13 con la opción de compartir crea una dirección pública que redirige al servidor local a través de sus propios servidores, que actúan solo como intermediarios, y que caduca a la semana.14 El enlace usa HTTPS, condición que exigen los navegadores para permitir el acceso al micrófono, y puede protegerse con usuario y contraseña. Cumple todos los requisitos salvo uno implícito, la disponibilidad permanente, que para una demostración académica no es imprescindible.

5.3. Arquitectura

La figura 12 muestra los componentes. El navegador captura el micrófono y envía fragmentos de audio de 0,2 segundos al servidor de Gradio, que los entrega a la sesión de transcripción activa y devuelve el texto acumulado y las métricas. Todo el servidor se ejecuta en WSL2, el mismo entorno de E2 y E3, porque en Windows Voxtral no llega a tiempo real, como se verá en la sección 5.5.

10 LocalVocal. Disponible en: https://github.com/locaal-ai/obs-localvocal (consulta: 19-10-2026).
11 Documentación de las GPU de Spaces. Disponible en: https://huggingface.co/docs/hub/spaces-gpus (consulta: 19-10-2026).
12 Documentación de ZeroGPU. Disponible en: https://huggingface.co/docs/hub/spaces-zerogpu (consulta: 19-10-2026).
13 Gradio. Disponible en: https://www.gradio.app/ (consulta: 19-10-2026).
14 Guía de Gradio para compartir aplicaciones. Disponible en: https://www.gradio.app/guides/sharing-your-app (consulta: 19-10-2026).

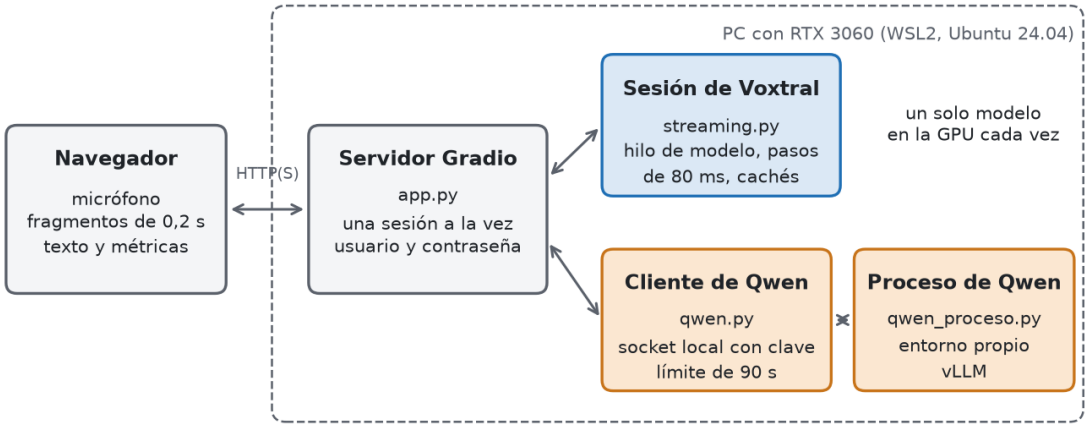


Figura 12. *Arquitectura del demostrador. Voxtral se ejecuta dentro del proceso de la aplicación; Qwen3-ASR, en un proceso aparte con su propio entorno. Elaboración propia.*

Dos decisiones de diseño se derivan directamente del capítulo 4. La primera es que los dos modelos no conviven en la GPU: Voxtral ocupa unos 8,25 GiB y vLLM reserva el 88 % de la memoria para Qwen3-ASR, de modo que cambiar de modelo significa descargar uno y cargar el otro. La segunda es que solo hay una sesión de transcripción a la vez. Con un margen del 11 al 16 % frente al tiempo real, dos sesiones simultáneas de Voxtral se retrasarían las dos.

5.4. Implementación

5.4.1. La sesión de streaming de Voxtral

El núcleo del demostrador es la clase Sesion del módulo streaming.py. Su diseño parte de una restricción de la implementación de transformers: la generación en streaming de Voxtral se invoca una sola vez con generate, que recibe los rasgos acústicos a través de un generador de Python y va pidiendo el siguiente trozo a medida que lo necesita. El modelo no espera audio: lo reclama. El micrófono, en cambio, entrega audio cuando quiere. Una cola une los dos ritmos.

Cada sesión crea un hilo dedicado que ejecuta generate. El servidor de Gradio, al recibir un fragmento del navegador, lo deposita en la cola de la sesión y vuelve inmediatamente. El hilo del modelo, cuando el generador le pide el siguiente trozo, saca de la cola el audio que necesita, o espera si aún no ha llegado. El texto sale por el lado contrario mediante un TextIteratorStreamer de transformers, que el servidor lee sin bloquearse cada vez que el navegador envía audio nuevo. La figura 13 muestra el intercambio de mensajes durante una sesión.

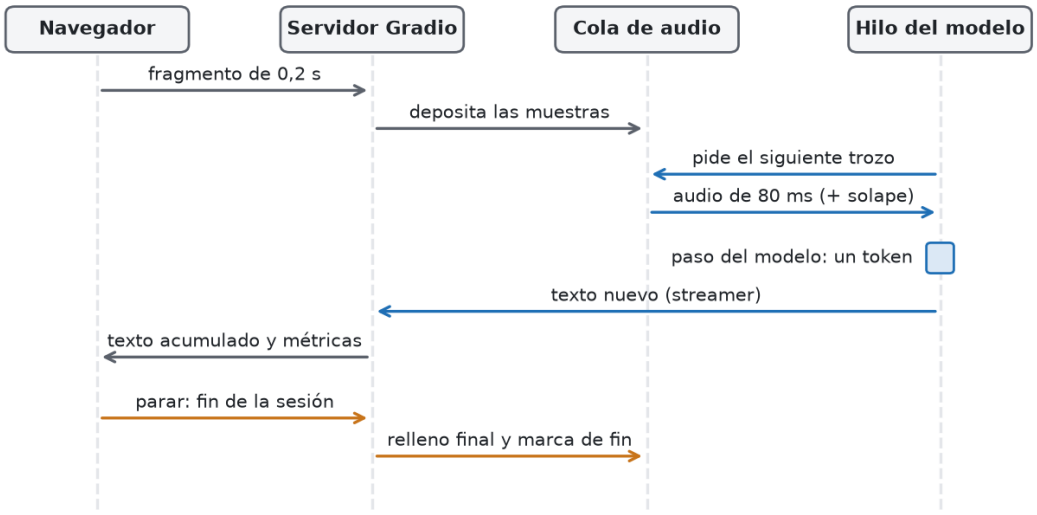


Figura 13. *Secuencia de una sesión de transcripción con Voxtral. Los mensajes del hilo del modelo se repiten cada 80 ms de audio; los del navegador, cada 0,2 s. Elaboración propia.*

```
def _trozos(self):
  while True:
    start = self.mel * hop - win // 2
    end = start + processor.num_samples_per_audio_chunk
    while end >= len(self.audio):
      if self.fin:
        return
      self._recibir()
    t = processor(self.audio[start:end], is_streaming=True,
            is_first_audio_chunk=False, return_tensors="pt")
    self.mel += processor.audio_length_per_tok
    yield t.to(model.device, dtype=model.dtype).input_features
```

El fragmento anterior es el generador que alimenta al modelo. En cada paso calcula qué tramo del audio recibido corresponde al siguiente trozo, de 105 ms, que se solapa con el anterior; si ese tramo aún no ha llegado, bloquea el hilo en la cola hasta que llegue o hasta que la sesión termine; después convierte el tramo en rasgos log-Mel con el procesador del modelo y avanza 8 tramas Mel, es decir, 80 ms. Es exactamente el mismo corte que usan los cuadernos de E2 y E3, y eso permitió comprobar que la aplicación reproduce sus resultados (sección 5.5). Al terminar la sesión se añade al final el relleno de silencio que el modelo necesita para emitir las últimas palabras, que todavía estaban dentro de su ventana de retardo.

El navegador suele capturar a 48 kHz y el modelo espera 16 kHz. El remuestreo se hace con la biblioteca soxr en modo de flujo,[15] que conserva el estado del filtro entre fragmentos y evita las discontinuidades que produciría remuestrear cada fragmento por separado.

15 python-soxr. Disponible en: https://github.com/dofuuz/python-soxr (consulta: 19-10-2026).

5.4.2. Qwen3-ASR en un proceso aparte

Integrar Qwen3-ASR tropezó con el mismo conflicto de versiones del capítulo 4: su paquete fija una versión de transformers incompatible con Voxtral, y su streaming exige vLLM. No pueden cargarse en el mismo proceso de Python. La solución fue ejecutarlo en un proceso hijo, lanzado con el intérprete de su propio entorno, y comunicarse con él por un socket local.

El módulo qwen.py abre un punto de escucha en la interfaz local con un puerto libre elegido por el sistema y una clave aleatoria de autenticación, lanza qwen_proceso.py pasándole ambos y espera a que el hijo se conecte y anuncie que el modelo está cargado. El protocolo es mínimo: un diccionario abre una sesión con el idioma y el tamaño de bloque; cada mensaje de bytes es un fragmento de audio, que el hijo pasa a streaming_transcribe; la cadena fin cierra la sesión; un valor nulo termina el proceso. El hijo responde con el texto acumulado y el tiempo de cómputo cada vez que se completa un bloque. La clase SesionQwen ofrece al resto de la aplicación la misma interfaz que Sesion, de modo que la interfaz no distingue qué modelo hay detrás.

Las lecciones del capítulo 4 se tradujeron en tres límites. Los bloques disponibles son de 1, 2 y 4 segundos; el de 0,48 s, que en E2 superó el tiempo real en las intervenciones largas, no se ofrece. Cada sesión con micrófono se corta a los 90 segundos, con un aviso, para no acercarse al techo de contexto. Y la configuración del motor reproduce la de los experimentos (88 % de la memoria y 24.576 tokens de contexto). El idioma puede fijarse o dejarse en detección automática.

5.4.3. Interfaz

La figura 14 muestra la interfaz durante la transcripción de una intervención de Europarl-ST. En la parte superior se elige el modelo y su parámetro de latencia: el retardo de Voxtral, entre 160 y 1200 ms y por defecto 480, o el bloque y el idioma de Qwen3-ASR. Los retardos de 160 y 320 ms se mantienen pese a su inestabilidad en E2, porque permiten ver en directo qué se pierde al bajar la latencia; el valor por defecto está dentro de la banda fiable.



Figura 14. *Interfaz del demostrador en la pestaña de audios de Europarl-ST, con la transcripción de Voxtral a 480 ms, la referencia del corpus y las métricas de la sesión. Elaboración propia.*

La pestaña del micrófono contiene el control de grabación, un botón para reiniciar y el cuadro de la transcripción, que se actualiza con cada fragmento. La de Europarl-ST ofrece ocho intervenciones del test, dos por idioma; al pulsar el botón de transcribir, el audio se reproduce en el navegador y, en paralelo, el servidor lo entrega a la sesión en fragmentos de 0,2 segundos al ritmo real, de modo que el texto avanza a la vez que la voz. Al terminar aparecen la referencia y el WER de la intervención. Bajo ambas pestañas, cuatro tarjetas muestran el retardo, el cómputo medio por paso, el RTF y el audio procesado.

Como solo hay una sesión a la vez, si otra persona está transcribiendo se muestra un aviso. Una sesión abandonada, por ejemplo al cerrar la pestaña sin detener la grabación, se libera tras 15 segundos sin recibir audio.

5.4.4. Concurrencia, sesiones y cambio de modelo

Gradio atiende cada evento del navegador en un hilo de trabajo, de modo que varias peticiones pueden llegar a la vez: fragmentos de audio de una sesión, el inicio de otra o una petición de cambio de modelo. El estado compartido es pequeño (la sesión activa, el modelo cargado y si hay una carga en curso) y se protege con un cerrojo. Abrir una sesión consiste en tomar el cerrojo, comprobar que no hay otra activa con actividad reciente ni una carga en marcha, terminar si existe la sesión abandonada y crear la nueva. Si la comprobación falla, el usuario recibe el aviso de que otra persona está usando el demostrador.

El cambio de modelo es la operación más delicada, porque dura minutos. Se implementa como un generador de Gradio que primero marca la carga en curso y desactiva el selector, lo que impide abrir sesiones mientras tanto, y después muestra el

aviso de que la carga puede tardar unos minutos. A continuación libera la memoria del modelo saliente (en el caso de Voxtral, eliminando la referencia a los pesos y vaciando la caché de PyTorch; en el de Qwen3-ASR, pidiendo al proceso hijo que termine) y carga el entrante. Si la carga falla, la aplicación vuelve a cargar Voxtral para no quedarse sin modelo. Un detalle de implementación evitó un bloqueo difícil de diagnosticar: el aviso no puede emitirse mientras se tiene el cerrojo, porque Gradio suspende el generador en ese punto y cualquier otra petición quedaría esperando a que la carga terminase.

Los dos modelos se calientan al cargarse. Voxtral transcribe diez segundos de silencio antes de aceptar la primera sesión, y el proceso de Qwen3-ASR ejecuta una transcripción breve con cada tamaño de bloque ofrecido. Las primeras llamadas a un modelo en la GPU incluyen inicializaciones que no se repiten, y sin el calentamiento las paga el primer usuario.

5.5. Pruebas

Las pruebas se diseñaron para responder a tres preguntas: si la aplicación transcribe igual que los cuadernos, si llega a tiempo real y si los mecanismos de control funcionan. La tabla 25 recoge los resultados.

Tabla 25. *Pruebas del demostrador sobre la RTX 3060.*

Prueba	**Resultado**
Equivalencia con el cuaderno de E3: ocho intervenciones, dos por idioma, a 480 ms	Texto idéntico en las 8
Entrada del micrófono a 48 kHz, simulada remuestreando una intervención	Mismo WER que el cuaderno tras el remuestreo con soxr
Cómputo por paso de Voxtral en Windows	Unos 87 ms por paso de 80 ms (RTF entre 1,08 y 1,14): no llega a tiempo real
Cómputo por paso de Voxtral en WSL2, con la aplicación sirviendo	Entre 68 y 76 ms (RTF entre 0,85 y 0,95)
Memoria de la GPU ocupada por Voxtral	8,25 GiB
Qwen3-ASR con bloques de 1 s sobre las ocho intervenciones, idioma fijado	Entre 214 y 260 ms por bloque (RTF entre 0,21 y 0,26); WER entre 2,10 y 13,27 %
Qwen3-ASR con detección automática del idioma	Una intervención en castellano reconocida como italiano: WER del 10,49 % frente al 4,32 % con el idioma fijado
Límite de 90 s de Qwen3-ASR con micrófono simulado	La sesión se corta a los 90 s y la aplicación queda libre
Cambio de modelo con los pesos fuera de la caché del sistema	541 s hacia Qwen3-ASR y 268 s hacia Voxtral
Voxtral tras volver de Qwen3-ASR	Mismo WER que antes del cambio

La primera fila es la más importante: la aplicación no es una reimplementación aproximada, sino el mismo cálculo que produjo los resultados del capítulo 4.

La medida del tiempo de cómputo deparó la sorpresa del desarrollo. En una primera versión, el cómputo por paso en WSL2 presentaba picos de entre 100 y 190 ms, por encima del presupuesto de 80 ms, en momentos irregulares. Concentrar todo el trabajo del modelo en un único hilo por sesión y limitar PyTorch a un hilo de procesador (torch.set_num_threads(1)) eliminó los picos y dejó el cómputo entre 68 y 76 ms, lo que apunta a una competencia por el procesador entre los hilos de cálculo de PyTorch y los del servidor web. En Windows, en cambio, ningún ajuste bajó de unos 87 ms por paso, y por eso el demostrador se ejecuta en WSL2. La captura de la figura 14 corresponde a la segunda transcripción tras arrancar la aplicación, con 73 ms por paso

y un RTF de 0,91. La primera transcripción de ese mismo arranque tardó 87 ms por paso, y en otro arranque distinto la primera, sobre una intervención francesa, tardó 79 ms, con un RTF de 0,99. La primera sesión tras arrancar parece pagar todavía costes de inicialización que el calentamiento del modelo no elimina del todo, y queda en el límite del tiempo real o ligeramente por encima.

La prueba de detección automática del idioma de Qwen3-ASR reproduce, en el otro modelo, el fallo que E1 encontró en Voxtral: sin la ayuda de saber en qué idioma escuchar, también Qwen3-ASR puede tomar el castellano por italiano. Por eso la interfaz propone el castellano por defecto en la pestaña del micrófono y fija el idioma de la intervención en la de Europarl-ST.

5.6. Limitaciones, riesgos y marco legal

El demostrador cumple su función, pero sus límites son los de una aplicación servida desde un ordenador personal. Funciona solo mientras el equipo está encendido y la aplicación arrancada; atiende a una persona cada vez; su enlace caduca a la semana y hay que regenerarlo; y el cambio de modelo, con los pesos en el disco mecánico en el que está instalado WSL2, tarda varios minutos. La tabla 26 recoge los riesgos de usarlo en una demostración y las medidas adoptadas.

Tabla 26. *Riesgos del demostrador y medidas adoptadas.*

Riesgo	Consecuencia	Medida
Equipo apagado o reiniciado	El enlace deja de funcionar	Arrancar la aplicación antes de la demostración y comprobar el enlace
Enlace caducado	El tutor o el tribunal no pueden acceder	Generar un enlace nuevo antes de cada prueba
Dos usuarios a la vez	Retraso de ambas sesiones	Sesión única con aviso y liberación tras 15 s de inactividad
Intervención larga con Qwen3-ASR	Techo de contexto, salida degenerada	Corte a los 90 s con aviso
Cambio de modelo durante la demostración	Varios minutos sin servicio	Seleccionar el modelo antes de empezar; aviso de carga en la interfaz
Acceso de terceros al enlace	Uso no autorizado de la GPU	Usuario y contraseña obligatorios para compartir
Truncamientos de Voxtral	Transcripción incompleta	Retardo por defecto de 480 ms, dentro de la banda fiable
Primera sesión tras arrancar más lenta	Retraso acumulado en la primera transcripción	Hacer una transcripción de prueba antes de la demostración

En cuanto a los datos, la voz de una persona la identifica, y el demostrador se diseñó para manejar la mínima cantidad posible de ella. El código de la aplicación procesa el audio del micrófono en memoria durante la sesión, no lo escribe en disco ni lo envía a ningún servicio externo distinto del intermediario del enlace de Gradio, que solo reenvía el tráfico, y lo descarta al terminar. Los ficheros temporales que pueda crear internamente la biblioteca Gradio no se han auditado. La aplicación se niega a arrancar en modo compartido si no se han definido un usuario y una contraseña.

En cuanto a la propiedad intelectual, los dos modelos se distribuyen con licencia Apache 2.0 (Mistral AI 2026; Shi et al. 2026), que permite su uso, modificación y redistribución con atribución. Europarl-ST se distribuye con licencia Creative Commons de reconocimiento no comercial (CC BY-NC 4.0) (Iranzo-Sánchez et al. 2020); el demostrador usa ocho de sus intervenciones con fines académicos, sin ánimo de lucro y citando su origen, dentro de lo que la licencia permite.

6. Conclusiones y trabajo futuro

6.1. Cumplimiento de los objetivos

El trabajo partía de una pregunta sencilla en apariencia: qué ocurre con un modelo fundacional de reconocimiento del habla cuando se le exige transcribir mientras se habla. La respuesta resultó depender menos de la calidad de cada modelo que de la forma en que cada uno entiende el tiempo. Se repasan a continuación los cuatro objetivos de la sección 1.2.

O1. Comportamiento en tiempo real. Cumplido. Se midieron la latencia algorítmica y computacional, el coste por paso, su evolución dentro de la intervención y la degradación frente al modo diferido de los dos modelos, primero en castellano y después en los cuatro idiomas. La conclusión principal es de arquitectura: un modelo causal con estado incremental, como Voxtral Realtime, mantiene constante el coste de cada paso y no pierde calidad al pasar a streaming, mientras que un modelo que reprocesa el audio acumulado, como Qwen3-ASR en su modo streaming, ve crecer su coste a lo largo de la intervención y choca con un techo de contexto tanto antes cuanto menor es la latencia pedida. A igual latencia algorítmica, y en el régimen por debajo del segundo que interesa para subtitular en directo, Voxtral fue el modelo adecuado; con latencias de varios segundos, Qwen3-ASR es una alternativa excelente. El objetivo dejó además una aportación de método: el factor de tiempo real no sirve para elegir un sistema en directo, y el uso del paso combinado con su crecimiento sí permite saber si un sistema sostiene el tiempo real de forma indefinida.

O2. Evaluación en diferido. Cumplido. Los dos modelos transcribieron las 729 intervenciones del test en los cuatro idiomas y mejoraron en todos ellos la línea base publicada con el corpus, con la salvedad de que esa línea base corresponde a la primera versión de Europarl-ST. Entre ellos, la precisión resultó equivalente una vez separados los fallos catastróficos, que son raros pero determinan el orden de los modelos en dos de los cuatro idiomas. Lo que los distingue en diferido es el tipo de fallo, no la calidad media.

O3. Demostrador. Cumplido. La aplicación transcribe en tiempo real la voz del usuario desde el navegador y las intervenciones de Europarl-ST con su referencia, permite elegir modelo y latencia, muestra el coste en directo y puede compartirse con acceso restringido sin coste alguno. Reproduce exactamente los resultados de los cuadernos y sostiene el tiempo real con Voxtral en la RTX 3060. Sus limitaciones (un único usuario, dependencia del equipo encendido, cambio de modelo lento) son las propias de una demostración servida desde un ordenador personal, y están documentadas.

O4. Fundamentos. Cumplido en la medida en que hizo falta para los demás. El estudio de la representación del audio, del Transformer, de los modelos fundacionales y del procesamiento incremental fue lo que permitió explicar los resultados en lugar de limitarse a describirlos: sin entender que Voxtral es causal, la igualdad exacta entre su modo diferido y su streaming habría parecido un error de medida.

6.2. Problemas, errores y lecciones aprendidas

Buena parte del tiempo del proyecto no se dedicó a experimentar, sino a conseguir que los experimentos pudieran ejecutarse. Los dos modelos exigen versiones incompatibles de la misma biblioteca, lo que obligó a mantener dos entornos en Windows y otros dos en WSL2; el streaming de Qwen3-ASR exige vLLM, que no funciona en Windows, lo que obligó a montar el entorno Linux; los entornos virtuales no pueden residir en el disco de Windows montado en WSL2, porque su sistema de archivos no admite algunas operaciones que necesita el instalador; una actualización de librosa eliminó la vía por la que se leían los audios del corpus; y el antivirus del equipo interceptaba las conexiones cifradas y hacía fallar las descargas de modelos y paquetes desde Python. Ninguno de estos problemas aparece en los resultados. Todos se resolvieron fijando versiones y documentando cada instalación, que es la lección más prosaica y quizá la más útil del trabajo.

El problema de más fondo fue el presupuesto de memoria de vLLM. El motor reserva la memoria antes de recibir ninguna petición y se niega a arrancar si no puede garantizar la longitud máxima de secuencia declarada; con el valor por defecto del modelo no arrancaba en la RTX 3060. Ajustar ese límite enseñó que el techo de contexto de Qwen3-ASR no es un fallo ocasional, sino una consecuencia directa de su forma de hacer streaming, y convirtió un problema de configuración en uno de los resultados del trabajo.

También hubo errores propios, y conviene dejarlos escritos porque cada uno tiene una forma sencilla de evitarse:

- Las primeras comparaciones de coste se hicieron con el factor de tiempo real y con los parámetros por defecto de cada modelo, que corresponden a latencias de 480 ms y de 2 segundos. No eran homogéneas. La revisión con el tutor que llevó a medir la latencia a igual régimen cambió las conclusiones del trabajo. Antes de comparar dos sistemas hay que fijar qué prometen.

- El uso del paso de Voxtral se calculó al principio sobre la ventana de 105 ms que procesa en cada paso, en lugar de sobre los 80 ms de audio nuevo que incorpora, lo que daba un margen frente al tiempo real mayor del real. Se detectó al revisar los datos para esta memoria y se corrigieron los documentos afectados. El denominador de una métrica de tiempo real es el audio nuevo, no el audio procesado.

- El retardo de 1920 ms se ensayó sin comprobar antes que no figuraba entre los admitidos por la ficha del modelo. Leer la especificación antes de diseñar el barrido lo habría evitado.

- El barrido de latencia usó las veinte primeras intervenciones de la lista, no una muestra aleatoria, y sus duraciones se tomaron de los audios ya rellenados con el silencio final que necesita Voxtral. El tercer experimento corrigió lo primero con un muestreo con semilla fija, y lo segundo se corrigió al redactar la memoria.

- Un primer intento del tercer experimento sin límite de duración dejó a Qwen3-ASR más de dos horas atascado en una sola intervención. Registrar los fallos sin

detener el barrido, y acotar la duración a la vista de los resultados anteriores, permitió completarlo.

En lo técnico, el trabajo obligó a aprender herramientas que no se habían usado durante el grado: la generación en streaming de la rama 5 de transformers, vLLM y su gestión de la memoria, WSL2, Gradio, el remuestreo en flujo con soxr y la comunicación entre procesos de Python con autenticación. Más que cualquiera de ellas, dejó un hábito: leer el código de una biblioteca cuando su documentación no basta. El reprocesado del audio acumulado de Qwen3-ASR, que explica la mitad de los resultados, no se descubrió en un artículo, sino en el código de su función de streaming.

En lo personal, la lección fue de paciencia y de desconfianza sana hacia las propias cifras. Varios de los resultados más interesantes aparecieron al preguntarse por qué un número no cuadraba: un inglés peor que el alemán, un WER del 154 %, dos modos de funcionamiento que daban exactamente el mismo texto. Ninguna de esas rarezas era un error, y todas enseñaban algo.

6.3. Relación del trabajo con los estudios cursados

El trabajo se apoya sobre todo en dos asignaturas optativas del bloque de Computación del Grado en Ingeniería Informática.

Percepción (11589). Sus unidades didácticas cubren el análisis de componentes principales, la optimización y el cálculo matricial, la probabilidad, la estadística y la teoría de la decisión, el análisis discriminante, la regresión logística binaria y multiclase, la regresión lineal, los perceptrones multicapa, el algoritmo de retropropagación, el aprendizaje profundo y los métodos basados en árboles. Casi todo el capítulo 2 descansa sobre ellas. La regla de decisión de Bayes es el punto de partida de la formulación del reconocimiento del habla; cada token que generan los dos modelos es la decisión de un clasificador multiclase cuya salida softmax generaliza la regresión logística; y la atención, las capas de alimentación hacia delante y el entrenamiento por gradiente de los Transformer son perceptrones multicapa y retropropagación a gran escala.

Aprendizaje automático (11594). Su temario incluye el descenso por gradiente, el algoritmo EM, el perceptrón multicapa y la retropropagación, las redes profundas y recurrentes, las máquinas de vectores soporte y los modelos gráficos. Las redes recurrentes y los modelos para secuencias son el antecedente directo de CTC, del transductor recurrente y de los modelos de codificador y decodificador del capítulo 2, y los modelos gráficos proporcionan el marco de los modelos ocultos de Markov sobre los que se construyó la línea base de Europarl-ST.

El trabajo movilizó también conocimientos más generales de la titulación: la programación y la gestión de entornos en Python, la concurrencia (el hilo del modelo y la cola de audio del demostrador) y la comunicación entre procesos, los sistemas operativos (WSL2 y la gestión de la memoria de la GPU) y el desarrollo de aplicaciones web.

En cuanto a las competencias transversales que evalúa el tribunal:

- **CT7, responsabilidad ética, medioambiental y profesional.** Se tuvo presente en el tratamiento de los datos del demostrador, que no almacena el

audio de los usuarios y exige credenciales; en el respeto de las licencias de los modelos y del corpus; y en el análisis del consumo energético del anexo C. Grado de desarrollo: medio.

- **CT8, comunicación efectiva.** La memoria, los documentos de observaciones revisados con el tutor y el propio demostrador, pensado para que un no especialista vea funcionar el sistema y su coste, son su ejercicio principal. Grado: alto.

- **CT9, pensamiento crítico.** Es la competencia que más trabajó el proyecto: cuestionar el factor de tiempo real como medida del coste en directo, separar los fallos catastróficos de la calidad media, corregir las métricas propias cuando no cuadraban. Grado: alto.

- **CT10, conocimiento de problemas contemporáneos.** La accesibilidad de los contenidos en directo y el acceso a debates públicos en otras lenguas son problemas actuales a los que el trabajo contribuye, como razona el anexo C. Grado: medio.

6.4. Trabajo futuro

Varias preguntas quedaron abiertas, y algunas tienen un camino claro:

- **Medir el coste real del tiempo real en Voxtral.** Su degradación al pasar a streaming es nula por construcción. Para saber cuánta calidad cuesta su causalidad habría que compararlo con Voxtral Mini, el modelo de la misma familia que procesa la grabación completa (Liu et al. 2025), con las mismas intervenciones y el mismo procedimiento.

- **Intervenciones largas en los cuatro idiomas.** El tercer experimento se limitó a 90 segundos por el techo de Qwen3-ASR. Repetirlo sin límite con Voxtral, y con Qwen3-ASR a bloques de 2 o 4 segundos, caracterizaría la mitad del corpus que quedó fuera.

- **La causa de los truncamientos de Voxtral.** Aparecen fuera de la banda de 480 a 960 ms y, ocasionalmente, dentro de ella. Analizar las probabilidades del token de fin en esos audios podría explicarlos y, quizá, evitarlos con una restricción en la decodificación.

- **Latencia percibida de extremo a extremo.** Las latencias medidas son las del modelo. En el demostrador se suman la captura en el navegador, los fragmentos de 0,2 segundos y la red; medirlas daría la latencia que realmente percibe el usuario.

- **Despliegue sin depender de un ordenador personal**, por ejemplo con una GPU en la nube que se suspenda cuando no se usa, si se dispone de financiación para ello.

Y hay caminos que, a la vista de los resultados, no conviene seguir. Usar Qwen3-ASR con bloques por debajo del segundo mientras su streaming reprocese todo el audio acumulado lleva a un sistema que se queda atrás en las intervenciones largas y que

acaba agotando su contexto. Y evaluar sistemas de transcripción en directo solo con el factor de tiempo real lleva, como se ha visto, a conclusiones equivocadas.

Bibliografía

ABID, A., ABDALLA, A., ABID, A., et al., 2019. Gradio: Hassle-Free Sharing and Testing of ML Models in the Wild. *arXiv preprint arXiv:1906.02569* [en línea], [consulta: 19 octubre 2026]. Disponible en: https://arxiv.org/abs/1906.02569.

BAEVSKI, A., ZHOU, H., MOHAMED, A., et al., 2020. wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations. *arXiv preprint arXiv:2006.11477* [en línea], [consulta: 19 octubre 2026]. Disponible en: https://arxiv.org/abs/2006.11477.

BOMMASANI, R., HUDSON, D.A., ADELI, E., et al., 2021. On the Opportunities and Risks of Foundation Models. *arXiv preprint arXiv:2108.07258* [en línea], [consulta: 19 octubre 2026]. Disponible en: https://arxiv.org/abs/2108.07258.

CHAN, W., JAITLY, N., LE, Q.V., et al., 2016. Listen, Attend and Spell: A Neural Network for Large Vocabulary Conversational Speech Recognition. *2016 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. Shanghai: IEEE, pp. 4960-4964. DOI 10.1109/ICASSP.2016.7472621.

CONNEAU, A., MA, M., KHANUJA, S., et al., 2023. FLEURS: Few-Shot Learning Evaluation of Universal Representations of Speech. *2022 IEEE Spoken Language Technology Workshop (SLT)*. Doha: IEEE, pp. 798-805. DOI 10.1109/SLT54892.2023.10023141.

DAVIS, S.B. y MERMELSTEIN, P., 1980. Comparison of Parametric Representations for Monosyllabic Word Recognition in Continuously Spoken Sentences. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 28, no. 4, pp. 357-366. DOI 10.1109/TASSP.1980.1163420.

GRAVES, A., 2012. Sequence Transduction with Recurrent Neural Networks. *arXiv preprint arXiv:1211.3711* [en línea], [consulta: 19 octubre 2026]. Disponible en: https://arxiv.org/abs/1211.3711.

GRAVES, A., FERNÁNDEZ, S., GOMEZ, F., et al., 2006. Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks. *Proceedings of the 23rd International Conference on Machine Learning (ICML)*. New York: ACM Press, pp. 369-376. DOI 10.1145/1143844.1143891.

IRANZO-SÁNCHEZ, J., SILVESTRE-CERDÀ, J.A., JORGE, J., et al., 2020. Europarl-ST: A Multilingual Corpus for Speech Translation of Parliamentary Debates. *ICASSP 2020 - 2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. Barcelona: IEEE, pp. 8229-8233. DOI 10.1109/ICASSP40776.2020.9054626.

JURAFSKY, D. y MARTIN, J.H., 2026. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition, with Language Models* [en línea]. 3.ª. Stanford: Stanford University. [consulta: 19 octubre 2026]. Disponible en: https://web.stanford.edu/~jurafsky/slp3/.

KWON, W., LI, Z., ZHUANG, S., et al., 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*. New York: ACM, pp. 611-626. DOI 10.1145/3600006.3613165.

LIU, A.H., EHRENBERG, A., LO, A., et al., 2025. Voxtral. *arXiv preprint arXiv:2507.13264* [en línea], [consulta: 19 octubre 2026]. Disponible en: https://arxiv.org/abs/2507.13264.

MACHÁČEK, D., DABRE, R. y BOJAR, O., 2023. Turning Whisper into Real-Time Transcription System. *Proceedings of the 13th International Joint Conference on Natural Language Processing and the 3rd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics: System Demonstrations*. Bali, Indonesia: Association for Computational Linguistics, pp. 17-24. DOI 10.18653/v1/2023.ijcnlp-demo.3.

MISTRAL AI, 2026. Voxtral Realtime. *arXiv preprint arXiv:2602.11298* [en línea], [consulta: 19 octubre 2026]. Disponible en: https://arxiv.org/abs/2602.11298.

NOROOZI, V., MAJUMDAR, S., KUMAR, A., et al., 2023. Stateful Conformer with Cache-based Inference for Streaming Automatic Speech Recognition. *arXiv preprint arXiv:2312.17279* [en línea], [consulta: 19 octubre 2026]. Disponible en: https://arxiv.org/abs/2312.17279.

PASZKE, A., GROSS, S., MASSA, F., et al., 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. *arXiv preprint arXiv:1912.01703* [en línea], [consulta: 19 octubre 2026]. Disponible en: https://arxiv.org/abs/1912.01703.

RABINER, L.R., 1989. A Tutorial on Hidden Markov Models and Selected Applications in Speech Recognition. *Proceedings of the IEEE*, vol. 77, no. 2, pp. 257-286. DOI 10.1109/5.18626.

RADFORD, A., KIM, J.W., XU, T., et al., 2023. Robust Speech Recognition via Large-Scale Weak Supervision. *Proceedings of the 40th International Conference on Machine Learning* [en línea]. S.l.: PMLR, pp. 28492-28518. [consulta: 19 octubre 2026]. Proceedings of Machine Learning Research, vol. 202. Disponible en: https://proceedings.mlr.press/v202/radford23a.html.

SHI, X., WANG, X., GUO, Z., et al., 2026. Qwen3-ASR Technical Report. *arXiv preprint arXiv:2601.21337* [en línea], [consulta: 19 octubre 2026]. Disponible en: https://arxiv.org/abs/2601.21337.

SHI, Y., WANG, Y., WU, C., et al., 2021. Emformer: Efficient Memory Transformer Based Acoustic Model for Low Latency Streaming Speech Recognition. *ICASSP 2021 - 2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. Toronto: IEEE, pp. 6783-6787. DOI 10.1109/ICASSP39728.2021.9414560.

STRUBELL, E., GANESH, A. y MCCALLUM, A., 2019. Energy and Policy Considerations for Deep Learning in NLP. *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Florencia: Association for Computational Linguistics, pp. 3645-3650. DOI 10.18653/v1/P19-1355.

VASWANI, A., SHAZEER, N., PARMAR, N., et al., 2017. Attention Is All You Need. *arXiv preprint arXiv:1706.03762* [en línea], [consulta: 19 octubre 2026]. Disponible en: https://arxiv.org/abs/1706.03762.

WOLF, T., DEBUT, L., SANH, V., et al., 2020. Transformers: State-of-the-Art Natural Language Processing. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Stroudsburg: Association for Computational Linguistics, pp. 38-45. DOI 10.18653/v1/2020.emnlp-demos.6.

YANG, A., LI, A., YANG, B., et al., 2025. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388* [en línea], [consulta: 19 octubre 2026]. Disponible en: https://arxiv.org/abs/2505.09388.

ZEGHIDOUR, N., KHARITONOV, E., ORSINI, M., et al., 2025. Streaming Sequence-to-Sequence Learning with Delayed Streams Modeling. *arXiv preprint arXiv:2509.08753* [en línea], [consulta: 19 octubre 2026]. Disponible en: https://arxiv.org/abs/2509.08753.

Anexo A. Glosario

Caché de claves y valores (caché KV). Memoria en la que un Transformer guarda las claves y los valores de los tokens ya procesados para no recalcularlos en cada paso de la generación. Su tamaño crece con la longitud de la secuencia.

Causal. Dicho de un modelo o de una capa: que en cada instante solo usa información del pasado y del presente, nunca del futuro.

Codificador y decodificador. Arquitectura en la que una red (codificador) transforma la entrada en una representación interna y otra (decodificador) genera la salida a partir de ella, token a token.

Crecimiento. Indicador propio de este trabajo: cociente entre la latencia computacional media del último tercio de los pasos de una intervención y la del primer tercio.

CTC (*connectionist temporal classification*). Clasificación temporal conexionista. Función de pérdida que permite entrenar un reconocedor sin alineaciones entre el audio y el texto, sumando sobre todas las alineaciones compatibles con la transcripción.

Diferido (modo). Transcripción de una grabación completa, disponible de antemano. En inglés, *offline*.

Espectrograma log-Mel. Representación del audio como una matriz de energías por banda de frecuencia en la escala Mel y por ventana de tiempo, en escala logarítmica.

HMM (*hidden Markov model*). Modelo oculto de Markov. Modelo probabilístico de estados que fue la base de los reconocedores del habla clásicos.

Latencia algorítmica. Espera impuesta por el diseño del sistema antes de poder decidir una palabra. No depende del hardware.

Latencia computacional. Tiempo que tarda el equipo en procesar cada paso. Depende del hardware y de la implementación.

Latencia total. Suma de la latencia algorítmica y la computacional: el retraso que percibe el usuario.

Modelo fundacional. Modelo entrenado con grandes volúmenes de datos amplios y adaptable a numerosas tareas posteriores. En el habla, SFM (*speech foundation model*).

Normalización. Transformación de la hipótesis y de la referencia (minúsculas, eliminación de la puntuación) antes de calcular el WER, para no contar como errores las diferencias de formato.

Paso. Cada llamada al modelo durante el streaming. En Voxtral Realtime incorpora 80 ms de audio nuevo; en Qwen3-ASR, un bloque.

Reconocimiento automático del habla (ASR, *automatic speech recognition*). Tarea de obtener automáticamente la transcripción de una señal de voz.

Retardo de transcripción. En Voxtral Realtime, tiempo que espera el modelo desde que termina una palabra hasta que la escribe. Se configura con transcription_delay_ms.

RNN-T (*recurrent neural network transducer*). Transductor recurrente. Arquitectura que combina una red acústica y una red de predicción sobre los símbolos ya emitidos.

RTF (*real-time factor*). Factor de tiempo real: tiempo de proceso dividido entre la duración del audio.

Streaming. Transcripción en tiempo real: el sistema recibe el audio por fragmentos a medida que se produce y emite texto sin esperar al final.

Techo de contexto. Longitud máxima de secuencia que un motor de inferencia puede atender con la memoria reservada. En Qwen3-ASR en streaming limita la duración de una intervención.

Token. Unidad mínima de texto con la que trabaja un modelo de lenguaje, normalmente un fragmento de palabra.

Transformer. Arquitectura de red neuronal basada en mecanismos de atención, sin recurrencia ni convoluciones.

Uso del paso. Indicador propio de este trabajo: porcentaje que la latencia computacional media representa sobre la duración del audio nuevo de cada paso.

vLLM. Motor de inferencia para modelos de lenguaje que gestiona la caché de claves y valores por páginas. Es el único con el que Qwen3-ASR admite streaming.

WER (*word error rate*). Tasa de error de palabra: sustituciones, eliminaciones e inserciones divididas entre el número de palabras de la referencia.

WSL2 (*Windows Subsystem for Linux*). Subsistema de Windows para Linux, que permite ejecutar una distribución Linux con acceso a la GPU desde Windows.

Anexo B. Presupuesto

El presupuesto valora el coste que habría tenido el trabajo realizado por un profesional contratado, con los recursos materiales efectivamente usados. Se compone de tres partidas: personal, amortización del equipo y energía. El software empleado es libre o de pesos abiertos (Python, PyTorch, transformers, vLLM, Gradio, Voxtral y Qwen3-ASR, estos dos con licencia Apache 2.0) y no tiene coste.

Personal. Se toma como referencia el XIX Convenio colectivo estatal de empresas de consultoría, tecnologías de la información y estudios de mercado y de la opinión pública (BOE-A-2025-7766), vigente de 2025 a 2027. Las tareas del trabajo (análisis, diseño, desarrollo y pruebas de software con poca experiencia previa) corresponden al área 3, «Desarrollo de software, programación y explotación de sistemas», grupo B, nivel 2, del artículo 15. Su retribución para 2026 en el anexo I del convenio es de 29.079,99 euros brutos anuales (26.753,59 de salario base y 2326,40 de plus de convenio), y la jornada máxima anual es de 1800 horas (artículo 20.1), lo que da 16,16 euros por hora. A esa cantidad se suman las cotizaciones a cargo de la empresa fijadas para 2026 por la Orden PJC/297/2026 (BOE-A-2026-7296): contingencias comunes, 23,60 % (artículo 4); desempleo con contrato indefinido, 5,50 %; Fondo de Garantía Salarial, 0,20 %; y formación profesional, 0,60 % (artículo 33.2); más el mecanismo de equidad intergeneracional, 0,75 % (artículo 16). En total, un 30,65 %. No se incluye la cotización por accidentes de trabajo, que depende de la actividad concreta de la empresa. Las horas son las 300 estimadas en el plan de trabajo de la sección 1.4. No se valora el tiempo del tutor.

Equipo. El ordenador usado (Intel Core i5-11400F, 16 GB de memoria, disco de estado sólido de 500 GB y NVIDIA GeForce RTX 3060 de 12 GB) se valora con el precio de un equipo comercial de esa misma configuración, 929 euros.16 Se amortiza al 25 % anual, el coeficiente lineal máximo que la Ley 27/2014 del Impuesto sobre Sociedades fija para los equipos para procesos de información (artículo 12.1.a; BOE-A-2014-12328, texto consolidado vigente), durante los siete meses del proyecto.

Energía. Se estima el consumo de la tarjeta gráfica durante las ejecuciones cuyo tiempo quedó registrado: 33,6 horas de E1 (1734 minutos de Voxtral y 284 de Qwen3-ASR) y 6,6 horas de E2 y E3, 40,2 horas en total. Con la potencia nominal de la RTX 3060, 170 W,17 suponen 6,83 kWh, que se valoran al precio medio de la electricidad para consumidores domésticos en España en el primer semestre de 2026 publicado por Eurostat, 0,2509 euros por kWh con impuestos.18 La cifra es una cota inferior: no incluye el resto del equipo, las cargas de los modelos, el desarrollo ni las pruebas del demostrador. La tabla 27 reúne las tres partidas.

16 Precio en PcComponentes del equipo «PcCom Gold Élite II Intel Core i5-11400F/16GB/500GB SSD/RTX 3060» según 3DJuegos, 17-02-2023. Disponible en: https://www.3djuegos.com/seleccion/noticias/baja-mil-euros-se-convierte-uno-pc-gaming-economicos-rtx-3060-12-gb-mucho-ojo-a-este-ordenador-a-precio-escandalo-pccomponentes (consulta: 19-10-2026).
17 Especificaciones de la GeForce RTX 3060. Disponible en: https://www.nvidia.com/es-es/geforce/graphics-cards/30-series/rtx-3060-3060ti/ (consulta: 19-10-2026).
18 Eurostat, conjunto de datos nrg_pc_204, consumidores domésticos con consumo anual de 2500 a 4999 kWh. Disponible en: https://ec.europa.eu/eurostat/databrowser/view/nrg_pc_204/default/table (consulta: 19-10-2026).

Tabla 27. *Presupuesto del trabajo*

Partida	**Cálculo**	**Importe (€)**
Salario	300 h × 16,16 €/h	4846,67
Cotizaciones a cargo de la empresa	30,65 % del salario	1485,50
Amortización del equipo	929 € × 25 % × 7/12	135,48
Energía de la GPU	6,83 kWh × 0,2509 €/kWh	1,71
Software	Libre o de pesos abiertos	0,00
Total		6469,36

El coste del trabajo es casi por completo coste de personal. La energía consumida por la GPU en los experimentos es irrelevante en términos económicos, aunque no lo es como orden de magnitud: el anexo C vuelve sobre ella.

Anexo C. Objetivos de Desarrollo Sostenible

Grado de relación del trabajo con los Objetivos de Desarrollo Sostenible (ODS), recogido en la tabla 28.

Tabla 28. *Grado de relación del trabajo con los ODS.*

Objetivos de Desarrollo Sostenible	Alto	Medio	Bajo	No procede
1. Fin de la pobreza				X
2. Hambre cero				X
3. Salud y bienestar				X
4. Educación de calidad		X		
5. Igualdad de género				X
6. Agua limpia y saneamiento				X
7. Energía asequible y no contaminante			X	
8. Trabajo decente y crecimiento económico				X
9. Industria, innovación e infraestructuras			X	
10. Reducción de las desigualdades	X			
11. Ciudades y comunidades sostenibles				X
12. Producción y consumo responsables			X	
13. Acción por el clima				X
14. Vida submarina				X
15. Vida de ecosistemas terrestres				X
16. Paz, justicia e instituciones sólidas		X		
17. Alianzas para lograr objetivos				X

Reflexión sobre la relación del trabajo con los ODS.

La relación más estrecha es con el ODS 10, reducción de las desigualdades. Entre sus metas figura potenciar y promover la inclusión social, económica y política de todas las personas, sin distinción, entre otras circunstancias, de discapacidad (meta 10.2).19 Para una persona sorda o con hipoacusia, el acceso a lo que se dice en una retransmisión, una clase o una sesión pública depende de que exista una transcripción, y su utilidad depende de que llegue a tiempo. Un subtítulo que aparece medio segundo después de la palabra permite seguir una conversación; uno que aparece al final del discurso, no. Por eso este trabajo no se ha limitado a medir cuánto se equivocan los modelos, sino cuánto tardan, y por eso el demostrador transcribe la voz en directo y no ficheros. El resultado principal, que un modelo causal ejecutado en una tarjeta gráfica doméstica sostiene la transcripción en tiempo real con una latencia de alrededor de medio segundo y un error en torno al 10 %, indica que el subtitulado automático en directo ya no requiere una infraestructura de servidor: está al alcance de una institución pequeña, de un centro educativo o de una asociación.

La relación con el ODS 16, paz, justicia e instituciones sólidas, es de grado medio y procede del escenario elegido. Dos de sus metas se refieren a garantizar decisiones inclusivas, participativas y representativas (16.7) y el acceso público a la información (16.10).20 Los debates de un parlamento multilingüe son públicos, pero seguirlos exige

19 Naciones Unidas, Objetivo 10. Disponible en: https://www.un.org/sustainabledevelopment/es/inequality/ (consulta: 19-10-2026).
20 Naciones Unidas, Objetivo 16. Disponible en: https://www.un.org/sustainabledevelopment/es/peace-justice/ (consulta: 19-10-2026).

entender la lengua del orador. La transcripción en tiempo real es el primer eslabón de la traducción simultánea automática, y los resultados del trabajo en cuatro lenguas, con sus diferencias entre idiomas, son una medida de lo cerca que está ese eslabón de funcionar sin intervención humana. El trabajo no traduce ni se ha desplegado en ninguna institución, y por eso la relación no se considera alta.

La relación con el ODS 4, educación de calidad, también es de grado medio. Las clases y conferencias son, como los debates, discurso oral prolongado, y su transcripción en directo facilita el acceso del alumnado con discapacidad auditiva y del que sigue la clase en una lengua que no domina. El demostrador permite precisamente eso, con la limitación de servir a una sola persona a la vez.

Con el ODS 9, industria, innovación e infraestructuras, la relación es baja: el trabajo no crea infraestructura, pero muestra que modelos de pesos abiertos, con licencias que permiten su uso y modificación, pueden ejecutarse en hardware asequible, lo que reduce la dependencia de servicios de pago.

Los ODS 7 y 12, energía asequible y no contaminante y producción y consumo responsables, merecen una reflexión aunque la relación sea baja, porque el aprendizaje profundo tiene un coste energético que no conviene ignorar. Se ha estimado que entrenar los grandes modelos de procesamiento del lenguaje exige recursos computacionales excepcionales, con un coste económico y ambiental considerable (Strubell, Ganesh y McCallum 2019). Este trabajo no entrena ningún modelo, y su propio consumo es modesto: la estimación del anexo B sitúa en unos 6,8 kWh el consumo de la tarjeta gráfica durante los experimentos registrados. Pero los resultados tienen una lectura energética. Un sistema que reprocesa todo el audio acumulado en cada paso analiza, en una intervención de dos minutos con bloques de 0,48 s, cerca de cien veces más audio que uno que procesa cada fragmento una sola vez (sección 3.3.2), y a esa latencia obtiene además un resultado peor. Elegir arquitecturas incrementales para el tiempo real no es solo una cuestión de latencia, sino también de consumo. La observación de que el factor de tiempo real puede llevar a elegir el sistema equivocado tiene, así, una segunda consecuencia: también puede llevar a elegir el que más energía gasta por palabra útil.

El resto de los ODS no guarda una relación identificable con el trabajo, que no aborda la pobreza, la alimentación, la salud, la igualdad de género, el agua, el empleo, las ciudades, el clima, los ecosistemas ni las alianzas entre actores.